

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет Факультет программной инженерии и компьютерной техники

Направление подготовки (специальность) Веб-технологии

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Тема задания: Meowle Network Lab.

Обучающийся Крылов В.А.
(Фамилия И.О.)

P4109

(номер группы)

Санкт-Петербург
2026 г.

1. ИСХОДНЫЕ ДАННЫЕ ЛАБОРАТОРНОЙ РАБОТЫ

В рамках лабораторной работы исследуется сетевое взаимодействие Android-приложения Meowle-lab.apk с учебным сервером Meowle. Приложение заранее настроено на работу с тестовым backend API, поэтому все действия выполняются только в пределах указанного учебного стенда. APK-файл мобильного приложения расположен в папке лабораторной работы. Работа с приложением выполняется в Android Emulator. Для перехвата HTTP-трафика используется Proxyman. Дополнительно для ручных HTTP-запросов и проверки API используется браузер и curl.

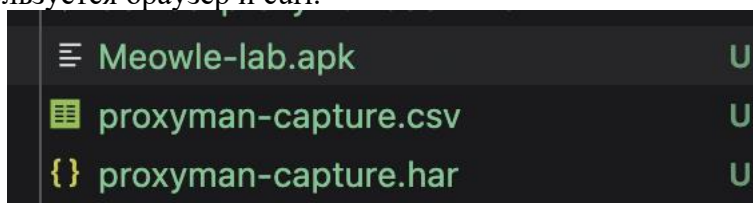


Рисунок 1 – Файл Meowle-lab.apk в папке лабораторной работы

1.2 Адреса учебного стенда

Таблица 1 – Адреса

Компонент	Адрес	Назначение
Frontend	http://144.31.255.0:8080	Веб-интерфейс учебного стенда
Backend API	http://144.31.255.0:3001	Основной backend API приложения
Swagger UI	http://144.31.255.0:3001/api-docs-ui/	Документация API
Version check	http://144.31.255.0:3001/version	Проверка версии стенда

Swagger UI используется для просмотра доступных методов API, параметров запросов и формата ответов: <http://144.31.255.0:3001/api-docs-ui/>.

Проверка версии стенда выполняется запросом: GET <http://144.31.255.0:3001/version>

Ожидаемый ответ: {"build": "lab-01"}

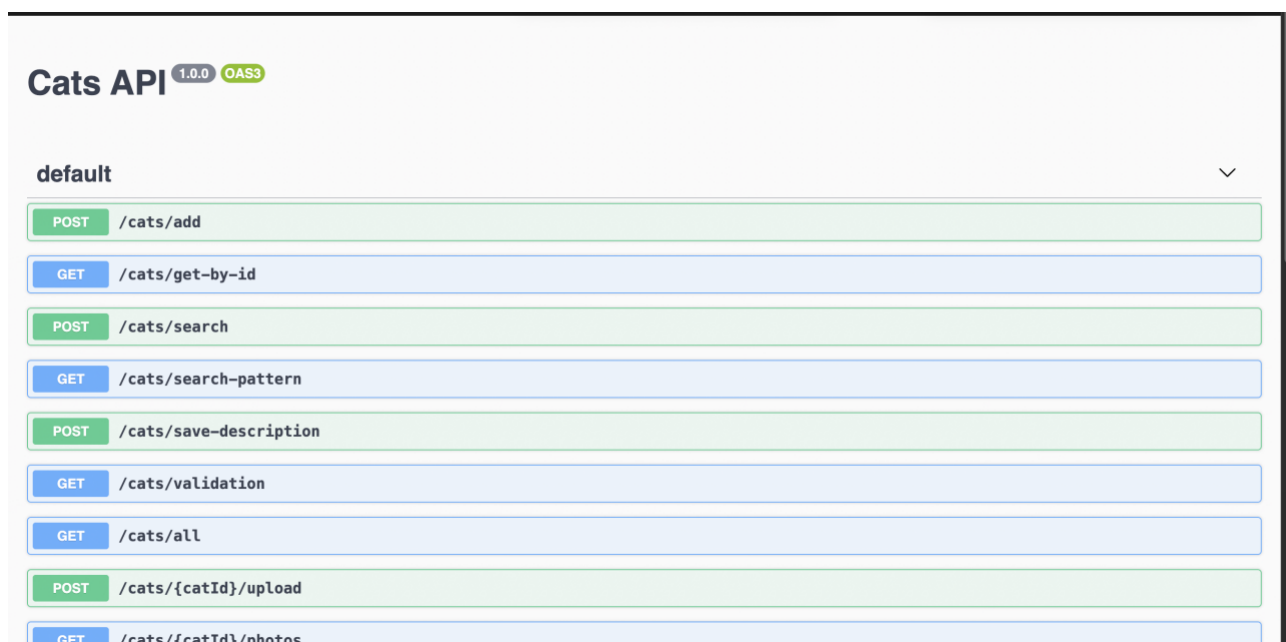


Рисунок 2 – Swagger UI учебного стенда

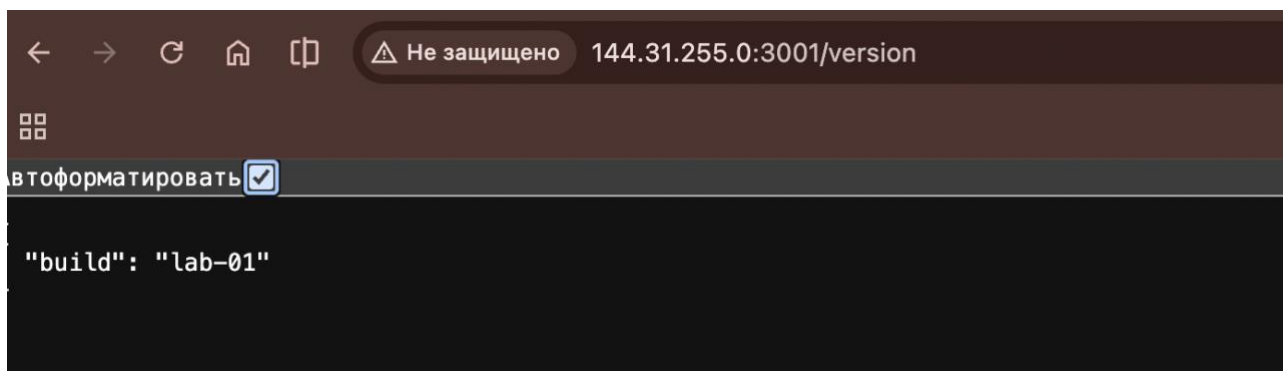


Рисунок 3 – Проверка версии стенда

1.3 Используемые инструменты

Таблица 2 – Инструменты

Инструмент	Назначение
Android Emulator	Запуск APK без физического Android-устройства
Android Studio	Создание и запуск Android Emulator
Proxyman	Перехват и анализ HTTP-запросов приложения
Браузер	Работа со Swagger UI и проверка адресов стенда
curl	Ручное выполнение HTTP-запросов
Android Studio Logcat	Дополнительная проверка запуска приложения и возможных ошибок

В данной работе выбран Android Emulator, так как физическое Android-устройство не используется его просто нету. Для перехвата трафика выбран Proxuman. Так как backend стенда работает по HTTP, установка сертификата Proxuman для базового перехвата не требуется.

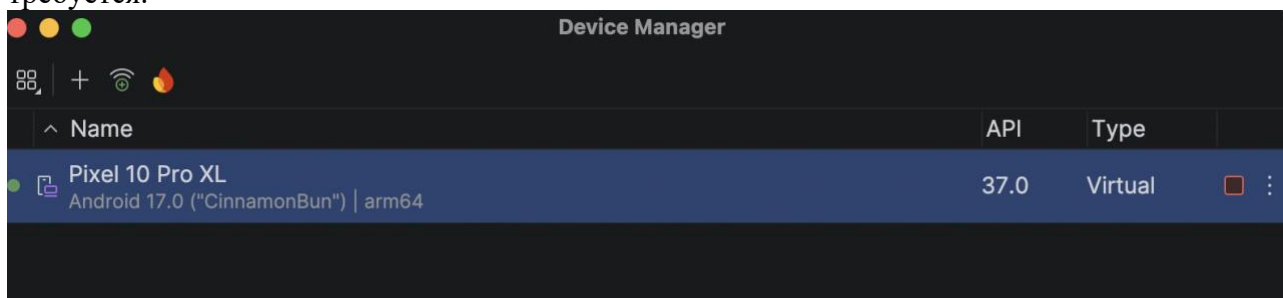


Рисунок 4 – Android Emulator в Android Studio

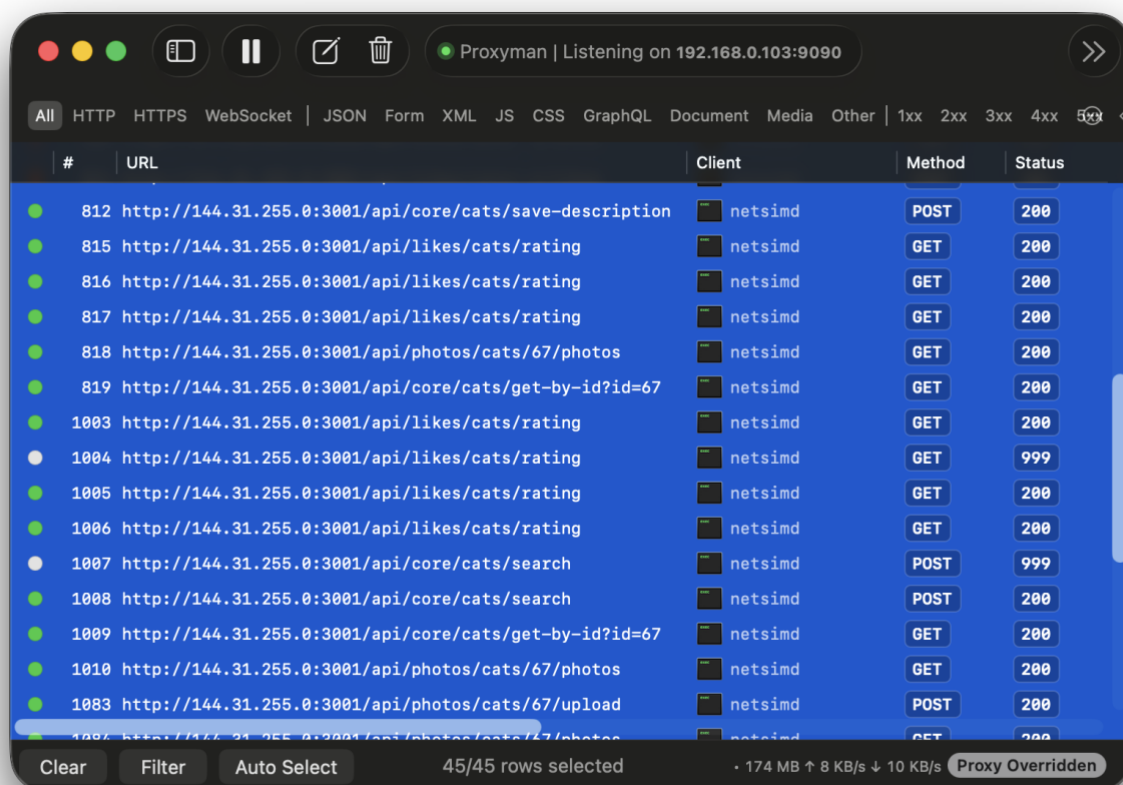


Рисунок 5 – Прохуман

1.4 Установка и запуск приложения

АРК-файл Meowle-lab.apk устанавливается на Android Emulator. После установки приложение запускается, и на экране входа используются валидные данные, указанные в задании.

Таблица 3 – Данные

Поле	Значение
Имя	Алексей
Телефон	+79990000000

После входа открывается вкладка рейтинга. Если в рейтинге отображаются имена котиков, значит приложение успешно подключилось к backend API учебного стенда.

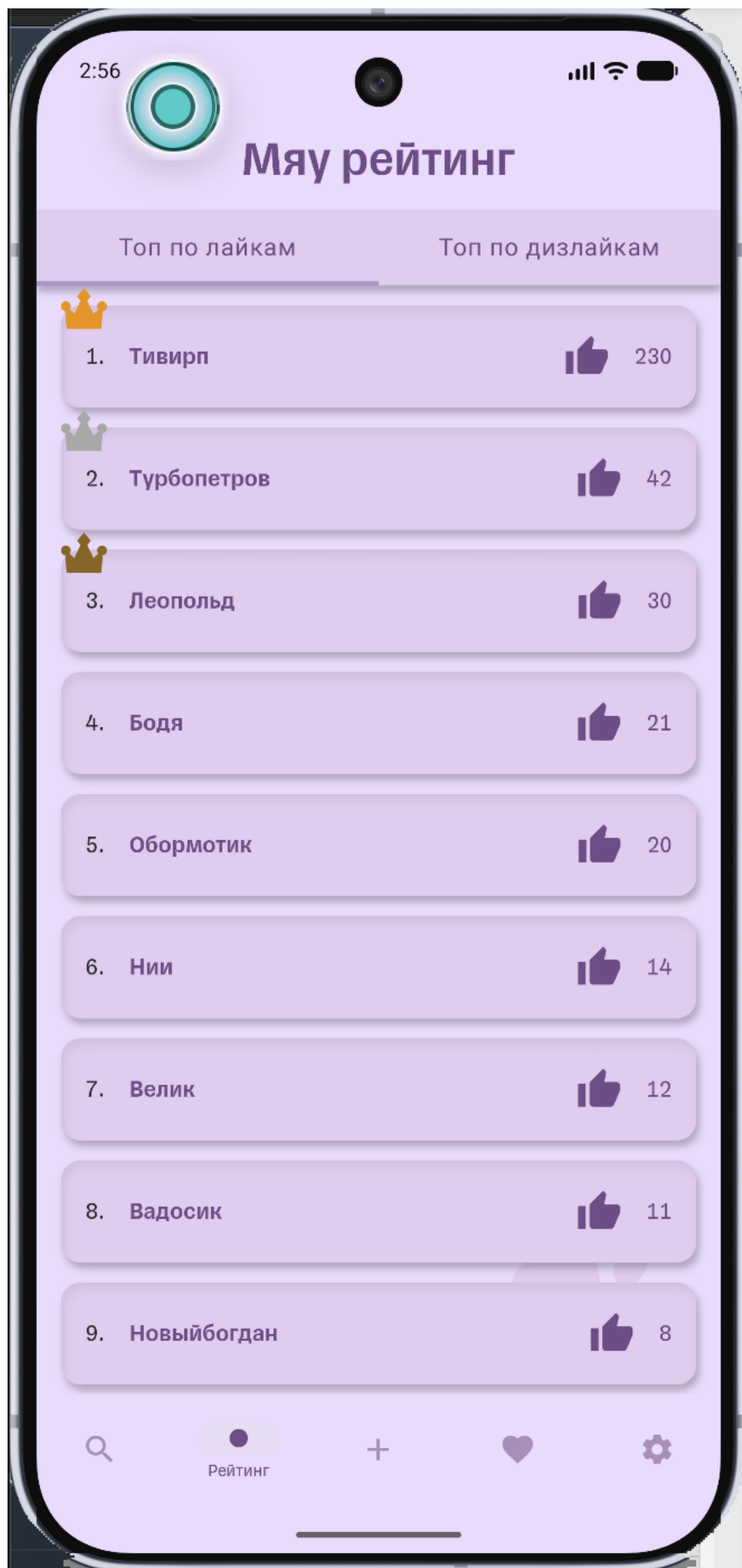


Рисунок 6 – Приложение Meowle в Android Emulator

1.5 Настройка прокси

Для перехвата сетевого трафика запускается Proxuman. В настройках Proxuman используется стандартный HTTP proxy port:

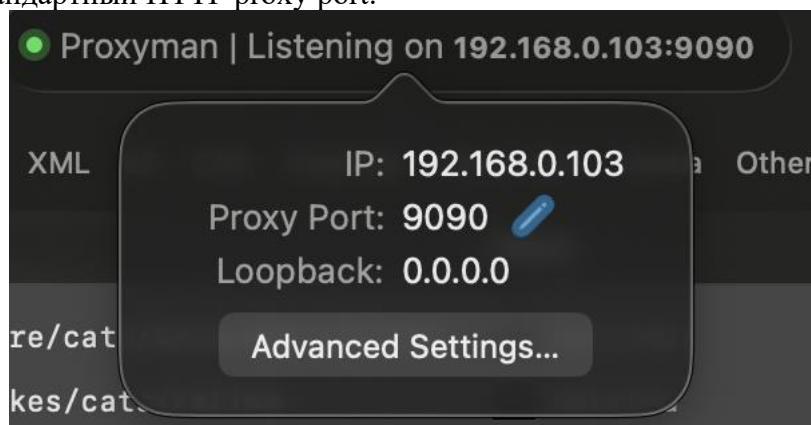


Рисунок 7 – HTTP proxy port

При работе с физическим Android-устройством в настройках Wi-Fi указывается ручной HTTP проху.

Таблица 4 - Значения

Поле	Значение
Host	IP-адрес компьютера в локальной сети
Port	9090

При работе с Android Emulator используется специальный адрес 10.0.2.2, который указывает из эмулятора на хостовую машину. Поэтому проху для эмулятора задается как: 10.0.2.2:9090

После настройки прокси запросы приложения к backend API начинают отображаться в Proxuman. Для удобства в Proxuman используется фильтр по адресу: 144.31.255.0 Так как стенд использует HTTP, сертификат Proxuman для базового перехвата не обязателен.

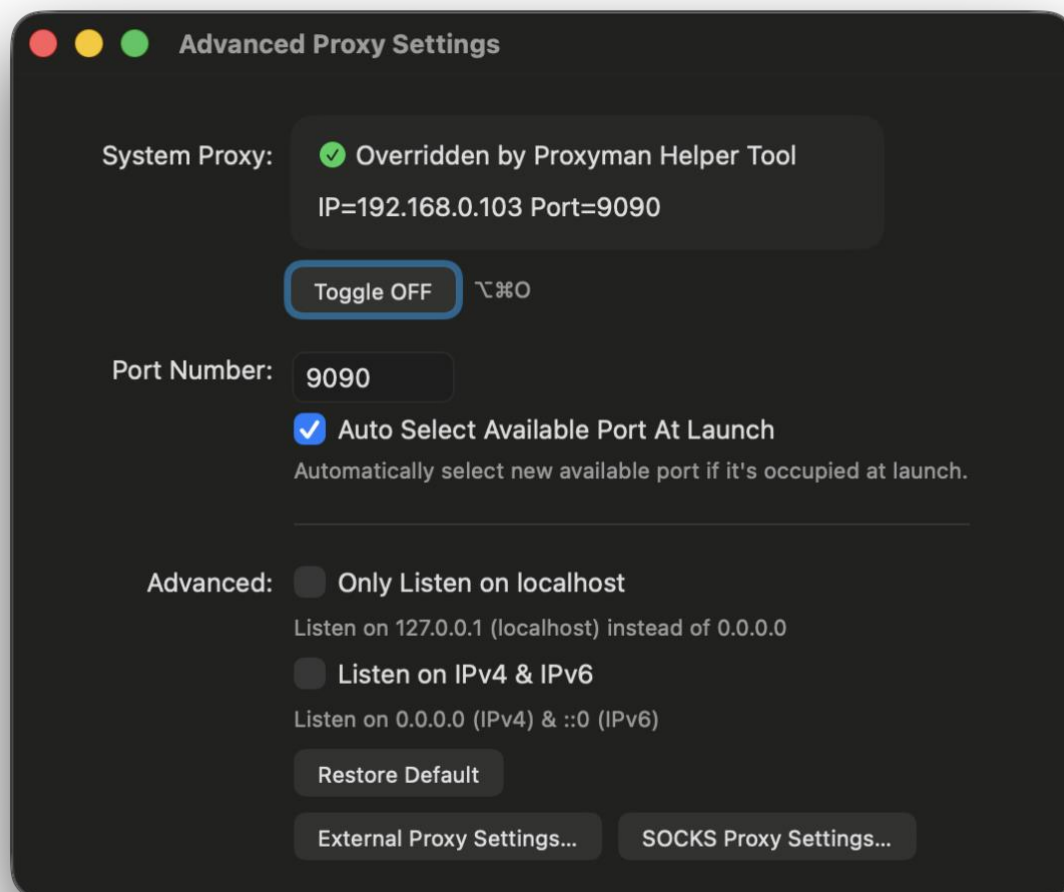


Рисунок 9 – Настройки порта Proxuman

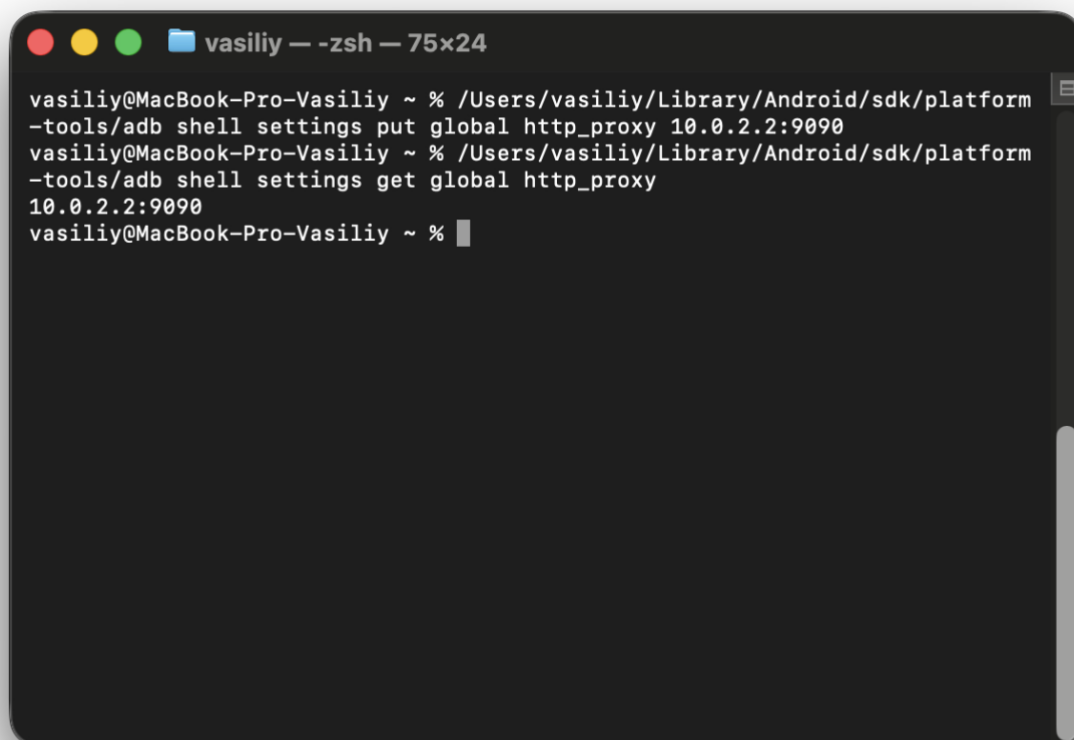


Рисунок 10 – Proxy 10.0.2.2:9090 в Android Emulator]

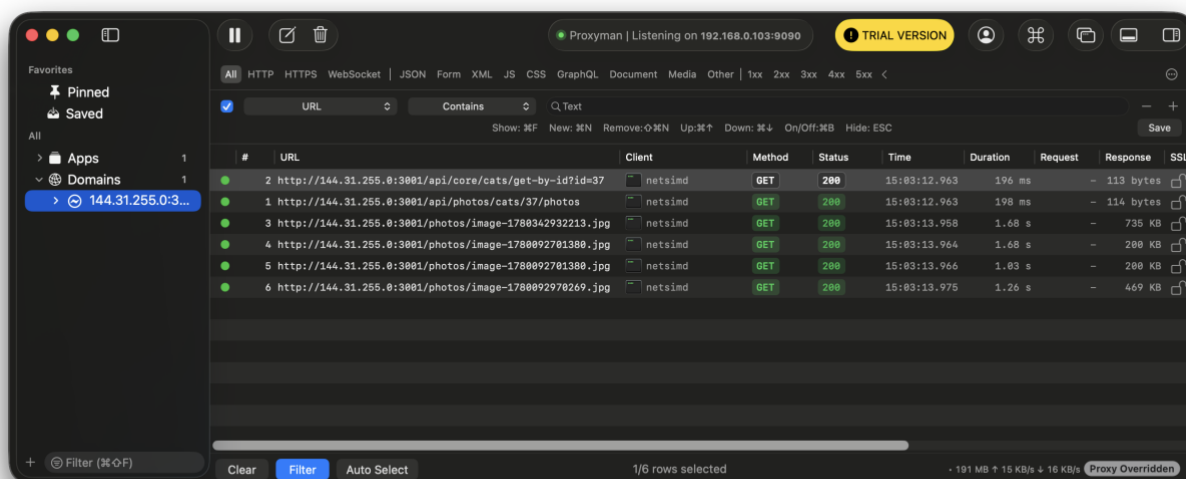


Рисунок 11 – Перехваченные запросы Meowle в Proxyman

1.6 Вывод по этапу подготовки

На этапе подготовки был определен состав учебного стенда, проверены адреса frontend, backend API и Swagger UI, установлен APK приложения Meowle в Android Emulator, выполнен вход с валидными учетными данными и настроен Proxyman для перехвата HTTP-трафика. После открытия вкладки рейтинга приложение успешно обращается к backend API, а запросы отображаются в Proxyman. Это подтверждает, что окружение готово для дальнейшей разведки API и анализа сетевого взаимодействия приложения.

2. РАЗВЕДКА API ЧЕРЕЗ SWAGGER

2.1 Цель этапа

Цель этапа - изучить документацию backend API через Swagger UI, найти методы, которые использует мобильное приложение Meowle, а также определить дополнительные web API-методы, доступные на учебном стенде. Swagger UI открыт по адресу: <http://144.31.255.0:3001/api-docs-ui/>



Рисунок 12 – Главная страница Swagger UI.

2.2 Порядок разведки API

Разведка API выполнялась по следующему плану:

Открыть Swagger UI в браузере.

Просмотреть список доступных endpoint-ов.

Разделить методы на две группы:

- mobile API, которые использует Android-приложение;
- web API, которые доступны через backend, но не обязательно используются в

основном мобильном сценарии.

Для каждого метода определить:

- HTTP method;
- path;
- назначение;
- параметры;
- пример запроса;
- пример ответа.

Отдельно отметить методы для поиска, карточки, добавления, описания, лайков, рейтинга, фото и удаления.

2.3 Найденные mobile API-методы

Основной мобильный API расположен в путях, начинающихся с /api/. Эти методы были подтверждены позднее в Proxuman HAR при работе Android-приложения.

Таблица 4 – Методы

Meth od	Path	Назначе ние	Параметры	Пример запроса
POS T	/api/core/cats/search	Поиск котиков	JSON: name, gender, o rder	{"gender":"unisex","name":"КотикТестовВаська","order":"asc"}
GET	/api/core/cats/get-by-id	Просмот р карточк и по id	Query: id	GET /api/core/cats/get-by-id?id=67
POS T	/api/core/cats/add	Добавле ние котика	JSON: массив cats, поля name, gender, des cription	{"cats":[{"name":"КотикТестовВаська","gender":"unisex","description":"КотикТестовВаська"}]}
POS T	/api/core/cats/save- description	Изменен ие описани я	JSON: catId, catDescri ption	{"catId":67,"catDescription":"КотикТестовВаська"}
POS T	/api/likes/cats/{catId}/l ikes	Лайк или дизлайк	Path: catId; JSON: like, dislike	POST /api/likes/cats/54/likes body {"like":true,"dislike":false}
GET	/api/likes/cats/rating	Рейтинг по лайкам и дизлайка м	Нет	GET /api/likes/cats/rating
GET	/api/photos/cats/{catId} /photos	Просмот р фото котика	Path: catId	GET /api/photos/cats/67/photos
POS T	/api/photos/cats/{catId} /upload	Загрузка фото	Path: catId; multipart file	POST /api/photos/cats/67/upload with file.jpg

POST

/cats/search

Поиск по имени и доп.характеристикам

Parameters

No parameters

Cancel

Request body required

application/json

Фильтр поиска имени

Examples:

[Modified value]

```
{
  "name": "КотикТестовБасяка",
  "gender": "unisex"
}
```

Execute

Clear

Responses

Curl

```
curl -X POST "http://144.31.255.0:3001/cats/search" -H "accept: application/json" -H "Content-Type: application/json" -d '{"name":"КотикТестовБасяка","gender":"unisex"}"
```

Request URL

http://144.31.255.0:3001/cats/search

Server response

Code

Details

200

Response body

```
{
  "groups": [
    {
      "title": "К",
      "cats": [
        {
          "id": 67,
          "name": "КотикТестовБасяка",
          "description": "КотикТестовБасяка",
          "tags": null,
          "gender": "unisex",
          "likes": 0,
          "dislikes": 0
        }
      ],
      "count": 1
    }
  ],
  "count": 1
}
```

Download

Response headers

```
connection: keep-alive
content-length: 214
content-type: application/json; charset=utf-8
date: Fri, 05 Jun 2020 07:18:17 GMT
etag: W/"d6-1Q0W0d0a0d1D00ia/Abn1iv2Pk"
x-powered-by: Express
```

Responses

Code

Description

Links

200

Имена по группам алфавита с их количеством

No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
{
  "groups": [
    {
      "title": "string",
      "count": 0,
      "cats": [
        {
          "id": 0,
          "name": "string",
          "description": "string",
          "tags": "string",
          "gender": "male",
          "likes": 0,
          "dislikes": 0
        }
      ],
      "count": 1
    }
  ],
  "count": 1
}
```

Рисунок 13 – Метод поиска котиков /api/core/cats/search в Swagger

GET /cats/get-by-id

Получение кота по id

Parameters

Name	Description
id required number (query)	Id кота

67

Execute Clear

Responses

Curl

curl -X GET "http://144.31.255.0:3001/cats/get-by-id?id=67" -H "accept: application/json"

Request URL

http://144.31.255.0:3001/cats/get-by-id?id=67

Server response

Code	Details
200	Response body <pre>{ "cat": { "id": 67, "name": "КотикТестовБася", "description": "КотикТестовБася", "tags": null, "gender": "unisex", "likes": 0, "dislikes": 0 } }</pre> Download Response headers <pre>connection: keep-alive content-length: 165 content-type: application/json; charset=utf-8 date: Fri, 05 Jun 2026 07:12:22 GMT etag: W/"a5-y1nVwUP3p2yS2Y9JrIkCQ94TLI" x-powered-by: Express</pre>

Responses

Code	Description	Links
200	Объект кота	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "cat": {
    "id": 0,
    "name": "string",
    "description": "string",
    "tags": [
      "string"
    ],
    "gender": "male",
    "likes": 0,
    "dislikes": 0
  }
}
```

Рисунок 14 – Метод просмотра карточки /api/core/cats/get-by-id

12

POST

/cats/add

Добавление списка имен

Parameters

Try it out

No parameters

Request body required

application/json

Список с именами и полом

Examples: [Modified value]

Example Value | Schema

```

{
  "cats": [
    {
      "name": "string",
      "gender": "male",
      "description": "string"
    }
  ]
}

```

Responses

Code	Description	Links
200	Список добавленных котов Media type application/json Controls Accept header. Example Value Schema <pre> { "cats": [{ "id": 0, "name": "string", "description": "string", "tags": ["string"], "gender": "male", "likes": 0, "dislikes": 0 }] } </pre>	

 No links |

Рисунок 16 - Методы добавления описания

POST

/cats/save-description

Сохранение описания имени

Parameters

Try it out

No parameters

Request body

required

application/json

Фильтр поиска имени

Examples:

Example Value | Schema

```
{
  "catId": 0,
  "catDescription": "string"
}
```

Responses

Code	Description	Links
200	Имена по группам алфавита с их количеством	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "description": "string",
  "tags": [
    "string"
  ],
  "gender": "male",
  "likes": 0,
  "dislikes": 0
}
```

Рисунок 17 – Методы изменения описания

POST

/api/likes/cats/{catId}/likes

Мобильное голосование за кота.

Parameters

Cancel

Name	Description
catId required integer (path)	catId

Request body required

application/json

Examples:

Modified value

```
{
  "like": true,
  "dislike": true
}
```

Execute

Responses

Code	Description	Links
200	Обновленный cat. Example Value Schema	No links

(no example available)

Рисунок 18 – Методы лайка, дизлайка

GET

/api/likes/cats/rating

Мобильный рейтинг по лайкам и дизлайкам.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X GET "http://144.31.255.0:3001/api/likes/cats/rating" -H "accept: */*"
```

Request URL

http://144.31.255.0:3001/api/likes/cats/rating

Server response

Code

Details

200

Response body

```
{
  "likes": [
    {
      "id": 54,
      "name": "Тимир",
      "description": "Описание изменено через прямой API-запрос",
      "tags": "",
      "gender": "unisex",
      "likes": 230,
      "dislikes": 1
    },
    {
      "id": 21,
      "name": "Турбонетрое",
      "description": "Новый текст",
      "tags": "",
      "gender": "unisex",
      "likes": 42,
      "dislikes": 40
    },
    {
      "id": 30,
      "name": "Леопольд",
      "description": "Леопольд вернулся. Удаляйтеааааа",
      "tags": "",
      "gender": "male",
      "likes": 30,
      "dislikes": 1
    }
  ]
}
```

Download

Response headers

```
connection: keep-alive
content-length: 2652
content-type: application/json; charset=utf-8
date: Fri, 05 Jun 2026 07:32:17 GMT
etag: W/"a5c-N8BcPF0i8V6x5ID9kac1IjNAvJo"
x-powered-by: Express
```

Responses

Code

Description

Links

200

Объект с массивами likes и dislikes.

Example Value

Schema

No links

Рисунок 19 – Метод рейтинга

16

GET /api/photos/cats/{catId}/photos

Мобильный список фотографий кота.

Parameters

Name

Description

catId required

integer (path)

67

Execute

Clear

Responses

Curl

curl -X GET "http://144.31.255.0:3001/api/photos/cats/67/photos" -H "accept: */*"

Request URL

http://144.31.255.0:3001/api/photos/cats/67/photos

Server response

Code

Details

200

Response body

{

"images": [

"/photos/image-1780634233797.jpg"

]

}

Download

Response headers

connection: keep-alive

content-length: 46

content-type: application/json; charset=utf-8

date: Fri, 05 Jan 2024 07:34:21 GMT

etag: W/"2e-9+voqrXCfcQr04y2iw0Jl3ClN84"

x-powered-by: Express

Responses

Code

Description

Links

200

Массив относительных ссылок images.

Example Value | Schema

(no example available)

No links

Рисунок 20 – методы просмотра фото

POST

/api/photos/cats/{catId}/upload

Мобильная загрузка фотографии кота.

Parameters

Name

Description

catId required

integer

(path)

67

Request body

multipart/form-data

file

string(\$binary)

Выберите файл

котика.jpeg

Execute

Clear

Responses

Curl

curl -X POST "http://144.31.255.0:3001/api/photos/cats/67/upload" -H "accept: */*" -H "Content-Type: multipart/form-data" -F "file=@котика.jpeg;type=image/jpeg"

Request URL

http://144.31.255.0:3001/api/photos/cats/67/upload

Server response

Code

Details

200

Response body

{
"fileUrl": "/photos/image-1786644980050.jpg"
}

Download

Response headers

connection: keep-alive
content-length: 45
content-type: application/json; charset=utf-8
date: Fri, 05 Jun 2020 07:36:20 GMT
etag: W/"2d-Frnj3Kt5pze15zqhdLXXYpxeQU"
x-powered-by: Express

Responses

Code

Description

Links

200

Относительная ссылка fileUrl.

Example Value

Schema

(no example available)

No links

Рисунок 21 – Метод загрузки фото

2.4 Найденные web API-методы

Кроме мобильных endpoints, Swagger показывает web API-методы без префикса /api. Они важны для лабораторной работы, потому что позволяют проверить отличие между мобильным сценарием и доступными backend-возможностями.

18

Таблица 5 – Методы

Method	Path	Назначени е	Параметры	Пример запроса	Пример ответа
POST	/cats/search	Поиск котиков через web API	JSON: name, gender	{"name":"Котик"}	{"groups":[...]}
GET	/cats/search-pattern	Поиск по началу имени	Query: name, limit	GET /cats/search-pattern?name=Ko&limit=5	{"moreResults":false,"cats":[..]}
GET	/cats/get-by-id	Получение карточки через web API	Query: id	GET /cats/get-by-id?id=67	{"cat":{"id":67,...}}
POST	/cats/add	Добавление котиков через web API	JSON: cats[]	{"cats":[{"name":"Котик","gender":"male"}]}	{"cats":[{"id":..., "name": "..."}]}
POST	/cats/save-description	Изменение описания через web API	JSON: catId, catDescription	{"catId":67,"catDescription":"Новое описание"}	{"id":67,"description":"Новое описание"}
GET	/cats/all	Получение списка всех котиков	Query: order, optional gender	GET /cats/all?order=asc	{"groups":[...],"count":...}
GET	/cats/validation	Получение правил валидации	Нет	GET /cats/validation	Список regex-правил
POST	/cats/{catId}/upload	Загрузка фото через web API	Path: catId; multipart file	POST /cats/67/upload	{"fileUrl":"/photos/..."}

GET	/cats/{catId}/photos	Просмотр фото через web API	Path: catId	GET /cats/67/photos	{"images":[...]}
POST	/cats/{catId}/like	Добавление лайка через web API	Path: catId	POST /cats/67/like	Увеличение счетчика лайков / ОК
DELETE	/cats/{catId}/like	Удаление лайка через web API	Path: catId	DELETE /cats/67/like	Уменьшение счетчика лайков / ОК
POST	/cats/{catId}/dislike	Добавление дизлайка через web API	Path: catId	POST /cats/67/dislike	Увеличение счетчика дизлайков / ОК
DELETE	/cats/{catId}/dislike	Удаление дизлайка через web API	Path: catId	DELETE /cats/67/dislike	Уменьшение счетчика дизлайков / ОК
DELETE	/cats/{catId}/remove	Удаление котика	Path: catId	DELETE /cats/67/remove	Ожидается удаление объекта
GET	/cats/likes-rating	Рейтинг по лайкам через web API	Нет	GET /cats/likes-rating	[{"name": "...", "likes": "...}]
GET	/cats/dislikes-rating	Рейтинг по дизлайкам через web API	Нет	GET /cats/dislikes-rating	[{"name": "...", "dislikes": "...}]
GET	/version	Проверка версии стенда	Нет	GET /version	{"build": "lab-01"}

Cats API 1.0.0 OAuth3

default

POST	/cats/add
GET	/cats/get-by-id
POST	/cats/search
GET	/cats/search-pattern
POST	/cats/save-description
GET	/cats/validation
GET	/cats/all
POST	/cats/{catId}/upload
GET	/cats/{catId}/photos
GET	/version
POST	/cats/{catId}/like
DELETE	/cats/{catId}/like
DELETE	/cats/{catId}/remove
POST	/cats/{catId}/dislike
DELETE	/cats/{catId}/dislike
GET	/cats/likes-rating
GET	/cats/dislikes-rating
POST	/api/core/cats/search
POST	/api/core/cats/add
GET	/api/core/cats/get-by-id
POST	/api/core/cats/save-description
GET	/api/likes/cats/rating
POST	/api/likes/cats/{catId}/likes
GET	/api/photos/cats/{catId}/photos
POST	<u>/api/photos/cats/{catId}/upload</u>

Рисунок 22 – web API методы /cats/ в Swagger

DELETE /cats/{catId}/remove

Удаления кота

Parameters Cancel

Name	Description
catId * required integer (path)	Id кота

66

Execute

LOADING

Responses

Code	Description	Links
200	Media type text/plain Controls Accept header. Example Value Schema OK	No links

Рисунок 23 - Метод удаления /cats/{catId}/remove

2.5. Методы, найденные по требованиям задания

По требованиям лабораторной работы необходимо было найти методы для основных действий.

Таблица 6 – Результаты

Требование	Найденный метод
Поиск котиков	POST /api/core/cats/search, POST /cats/search, GET /cats/search-pattern
Просмотр карточки по id	GET /api/core/cats/get-by-id?id={id}, GET /cats/get-by-id?id={id}
Добавление имени / котика	POST /api/core/cats/add, POST /cats/add
Изменение описания	POST /api/core/cats/save-description, POST /cats/save-description
Лайк в mobile API	POST /api/likes/cats/{catId}/likes c body {"like":true,"dislike":false}
Дизлайк в mobile API	POST /api/likes/cats/{catId}/likes c body {"like":false,"dislike":true}
Лайк через web API	POST /cats/{catId}/like, удаление лайка: DELETE /cats/{catId}/like
Дизлайк через web API	POST /cats/{catId}/dislike, удаление дизлайка: DELETE /cats/{catId}/dislike
Рейтинг	GET /api/likes/cats/rating, GET /cats/likes-rating, GET /cats/dislikes-rating
Просмотр фото	GET /api/photos/cats/{catId}/photos, GET /cats/{catId}/photos
Загрузка фото	POST /api/photos/cats/{catId}/upload, POST /cats/{catId}/upload
Удаление или изменение данных через web API	DELETE /cats/{catId}/remove, POST /cats/save-description

2.6 Наблюдения по Swagger

В Swagger видно, что backend содержит больше возможностей, чем основной мобильный сценарий. Например, через web API доступны отдельные методы удаления, получения всех записей, просмотра regex-правил валидации и управления лайками/дизлайками отдельными HTTP-методами.

Это важно для дальнейшего security-анализа: если метод не используется в интерфейсе приложения, это не значит, что он недоступен. Любой опубликованный backend endpoint можно вызвать напрямую через прокси, curl, Postman или Insomnia.

2.7. Вывод по этапу разведки API

На этапе разведки через Swagger UI была составлена карта API учебного стенда Meowle. Были найдены методы для поиска котиков, просмотра карточки, добавления записи, изменения описания, голосования, рейтинга, просмотра и загрузки фото, а также дополнительные web API-методы для изменения и удаления данных.

Полученная API-карта используется на следующих этапах лабораторной работы: при сопоставлении действий мобильного приложения с HTTP-запросами, при изменении запросов через replay и при поиске security-проблем.

3. КАРТА ДЕЙСТВИЙ ПРИЛОЖЕНИЯ

3.1. Цель этапа

Цель этапа - сопоставить действия пользователя в Android-приложении Meowle с реальными HTTP-запросами, которые приложение отправляет на backend API. Для перехвата трафика использовался Android Emulator и Proxyman.

Фильтр в Proxyman: 144.31.255.0. Основной источник данных для таблицы - HAR-экспорт Proxyman: proxyman-capture.har. В HAR видно, что Android-приложение использует mobile API с префиксом /api/. Web API-методы вида /cats/ в обычном мобильном сценарии не использовались, но остаются доступными для ручных проверок.

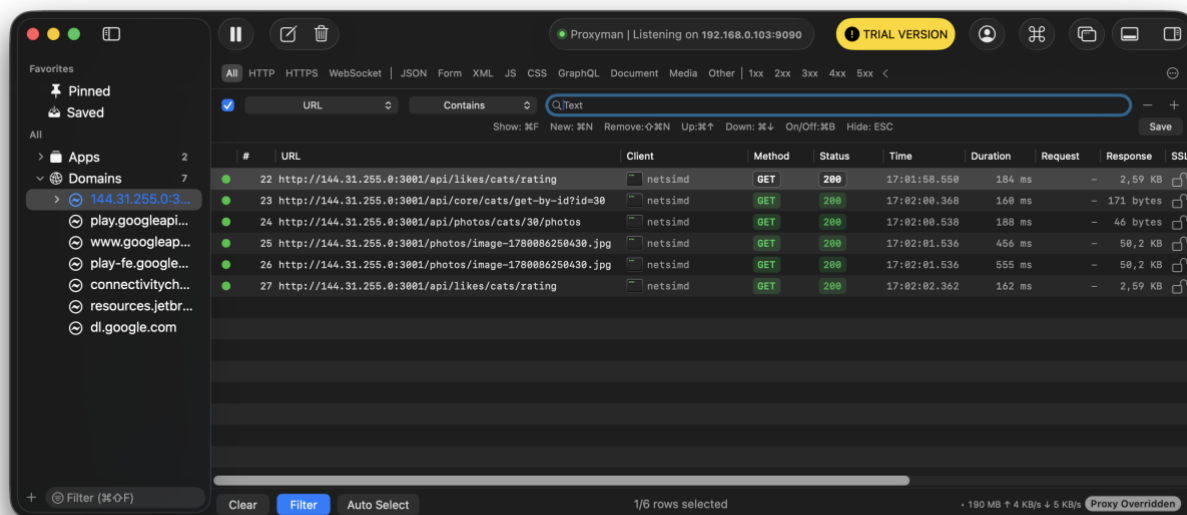


Рисунок 24 – Фильтр 144.31.255.0 в Proxyman и список запросов приложения

Таблица 6 – Действия приложения и HTTP-запросов

Действие в приложении	HTTP method	URL	Request body	Response code	Response body	Комментарий
Добавление нового котика	POST	http://144.31.255.0:3001/api/core/cats/add	{"cats":[{"description":"КотикТестовВаська","gender":"unisex","name":"КотикТестовВаська"}]}	200	{"cats":[{"id":67,"name":"КотикТестовВаська","description":"КотикТестовВаська","tags":"","gender":"unisex","likes":0,"dislikes":0}]}	Приложение создало запись, сервер нормализовал имя
Поиск котика	POST	http://144.31.255.0:3001/api/core/cats/search	{"gender":"unisex","name":"КотикТестовВаська","order":"asc"}	200	{"groups":[{"title":"К","cats":[{"id":67,"name":"КотикТестовВаська","description":"КотикТестовВаська","tags":"","gender":"unisex","likes":0,"dislikes":0}], "count":1}]}	Поиск вернул созданную запись id=67
Открытие карточки	GET	http://144.31.255.0:3001/api/core/cats/get-by-id?id=67	Нет	200	{"cat":{"id":67,"name":"КотикТестовВаська","description":"КотикТестовВаська","tags":"","gender":"unisex","likes":0,"dislikes":0}}	Карточка получается по прямому id
Изменение описания	POST	http://144.31.255.0:3001/api/core/cats/save-description	{"catDescription":"КотикТестовВаська","catId":67}	200	{"id":67,"name":"КотикТестовВаська","description":"КотикТестовВаська","tags":"","gender":"unisex","likes":0,"dislikes":0}	Описание изменяется отдельным запросом
Загрузка изображения	POST	http://144.31.255.0:3001/api/photos/cats/67/upload	multipart file, filename file.jpg, Content-Type: image/jpeg, Content-Length: 627862	200	{"fileUrl":"/photos/image-1780634233797.jpg"}	Файл загружен, сервер вернул путь к фото

Действие в приложении	HTTP method	URL	Request body	Response code	Response body	Комментарий
Просмотр фото после загрузки	GET	http://144.31.255.0:3001/api/photos/cats/67/photos	Нет	200	{"images":["/photos/image-1780634233797.jpg"]}	После загрузки фото появилось в списке
Лайк	POST	http://144.31.255.0:3001/api/likes/cats/54/likes	{"dislike":false,"like":true}	200	{"id":54,"name":"Тивирп","description":"Описание изменено через прямой API-запрос","tags":"","gender":"unisex","likes":230,"dislikes":0}	В mobile API лайк задается JSON-флагами
Дизлайк	POST	http://144.31.255.0:3001/api/likes/cats/54/likes	{"dislike":true,"like":false}	200	{"id":54,"name":"Тивирп","description":"Описание изменено через прямой API-запрос","tags":"","gender":"unisex","likes":230,"dislikes":1}	Используется тот же endpoint, но противоположные значения like/dislike
Открытие рейтинга	GET	http://144.31.255.0:3001/api/likes/cats/rating	Нет	200	{"likes":[...],"dislikes":[...]}	Ответ содержит два списка: рейтинг по лайкам и рейтинг по дизлайкам

3.2 Скриншоты запросов в Proxyman

Для подтверждения карты действий нужно приложить скриншоты request/response из Proxyman.

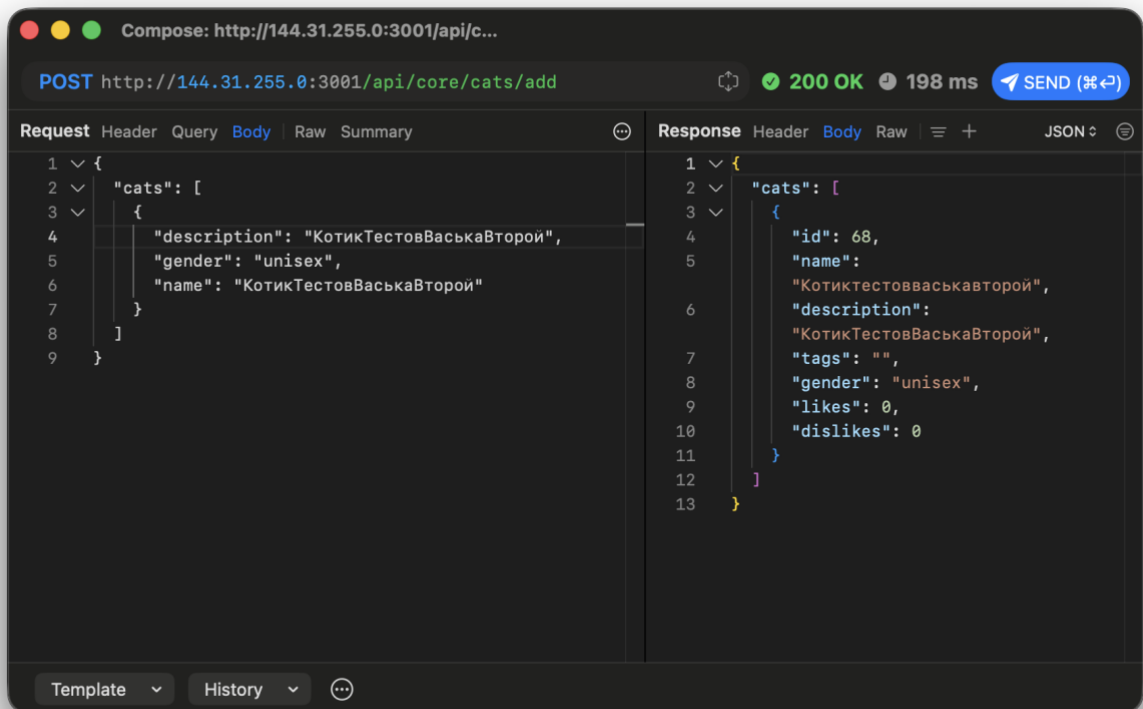


Рисунок 22 – Запрос добавления котика `/api/core/cats/add`

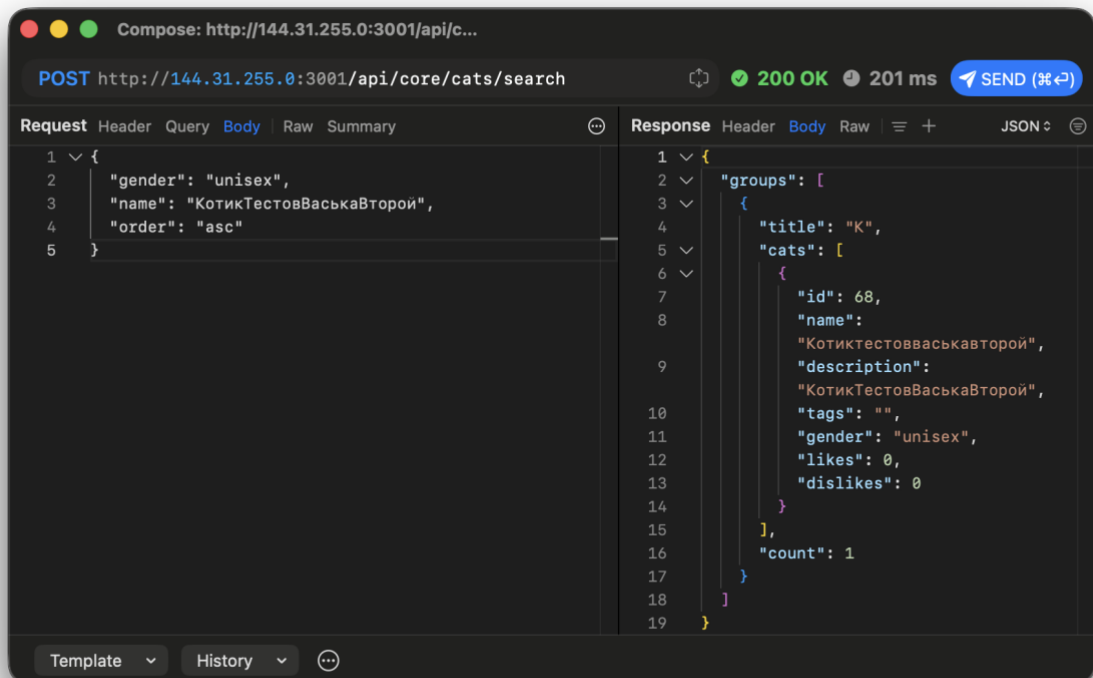


Рисунок 23 – запрос поиска `/api/core/cats/search`

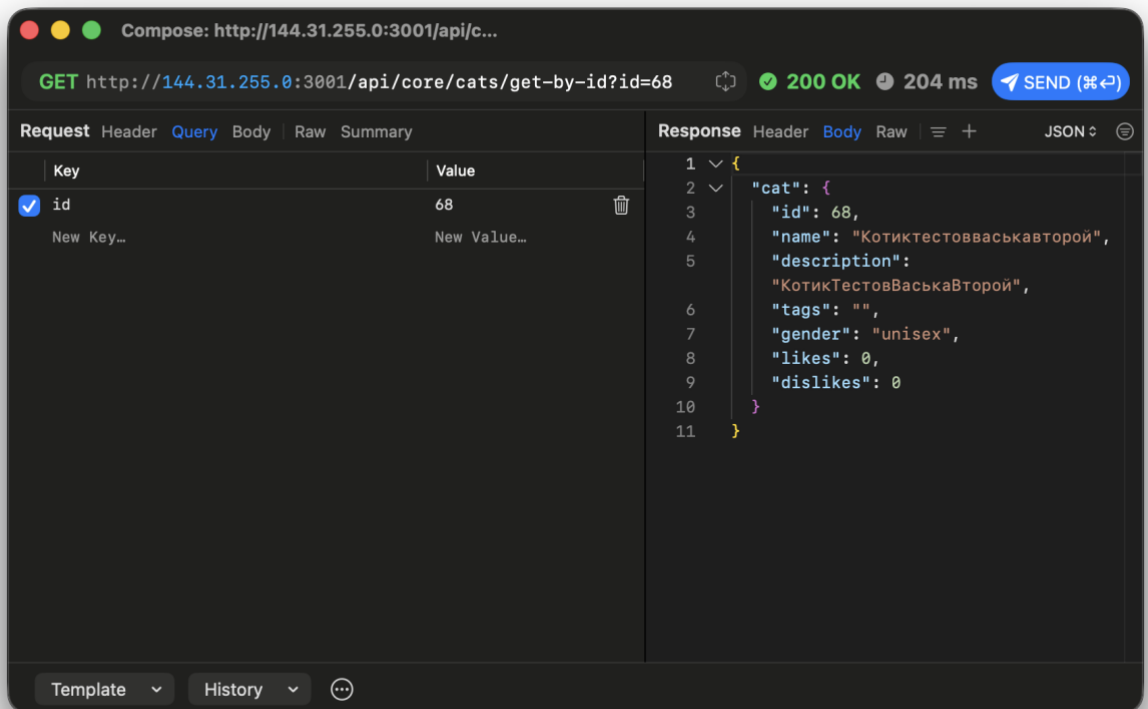


Рисунок 24 – Запрос открытия карточки `/api/core/cats/get-by-id`

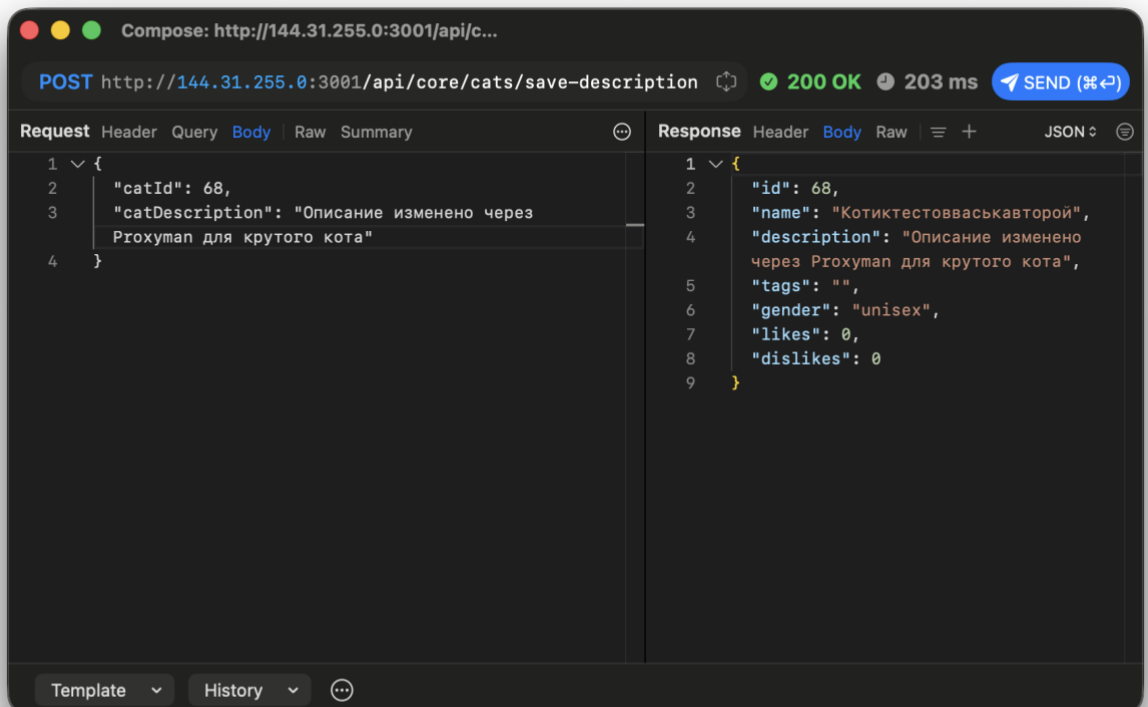


Рисунок 25 – Запрос изменения описания `/api/core/cats/save-description`

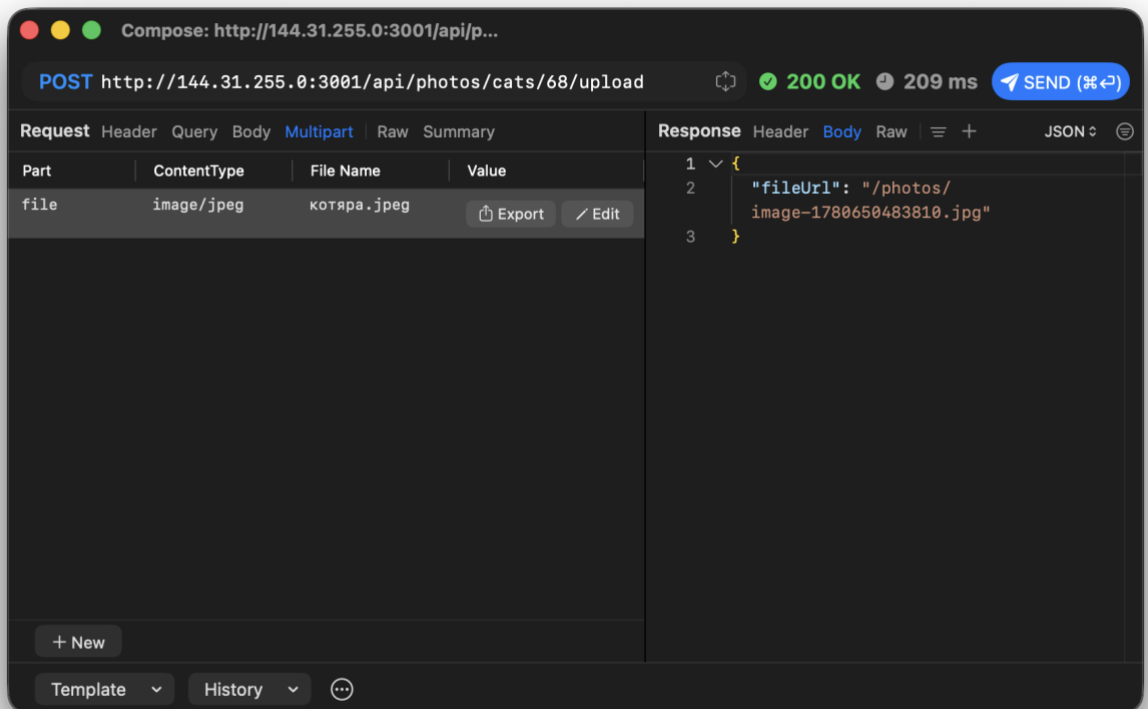


Рисунок 26 – Запрос загрузки изображения `/api/photos/cats/67/upload`

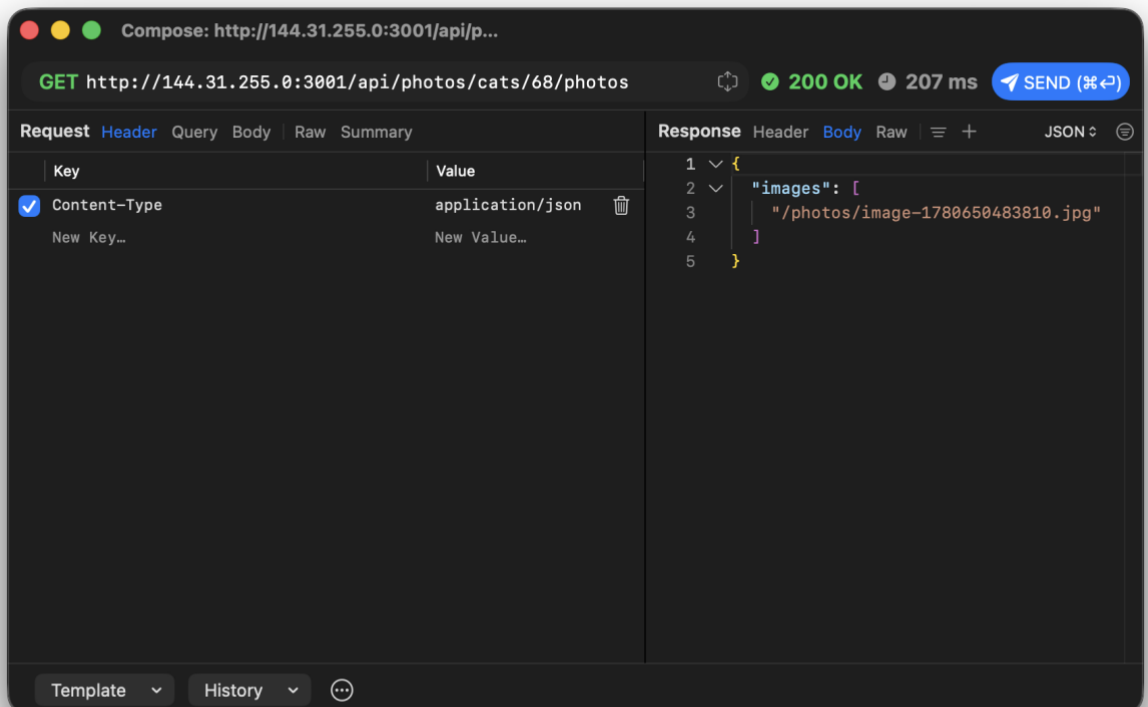


Рисунок 27 – Запрос списка фото `/api/photos/cats/68/photos`

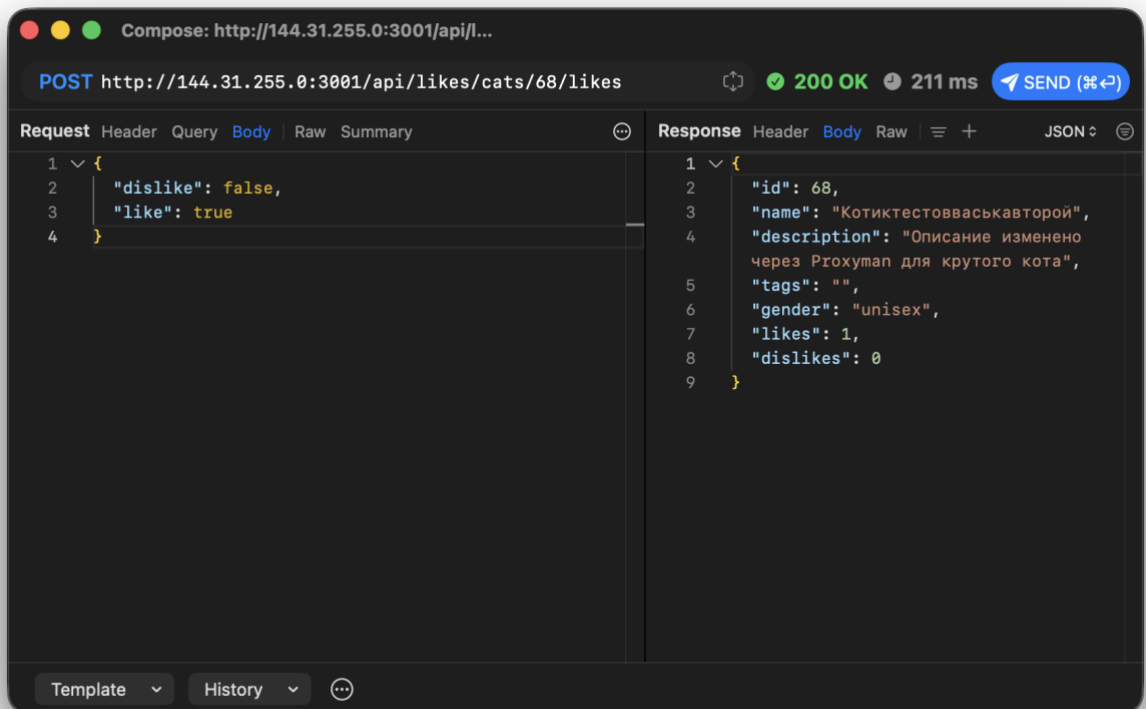


Рисунок 28 – Запрос лайка /api/likes/cats/68/likes

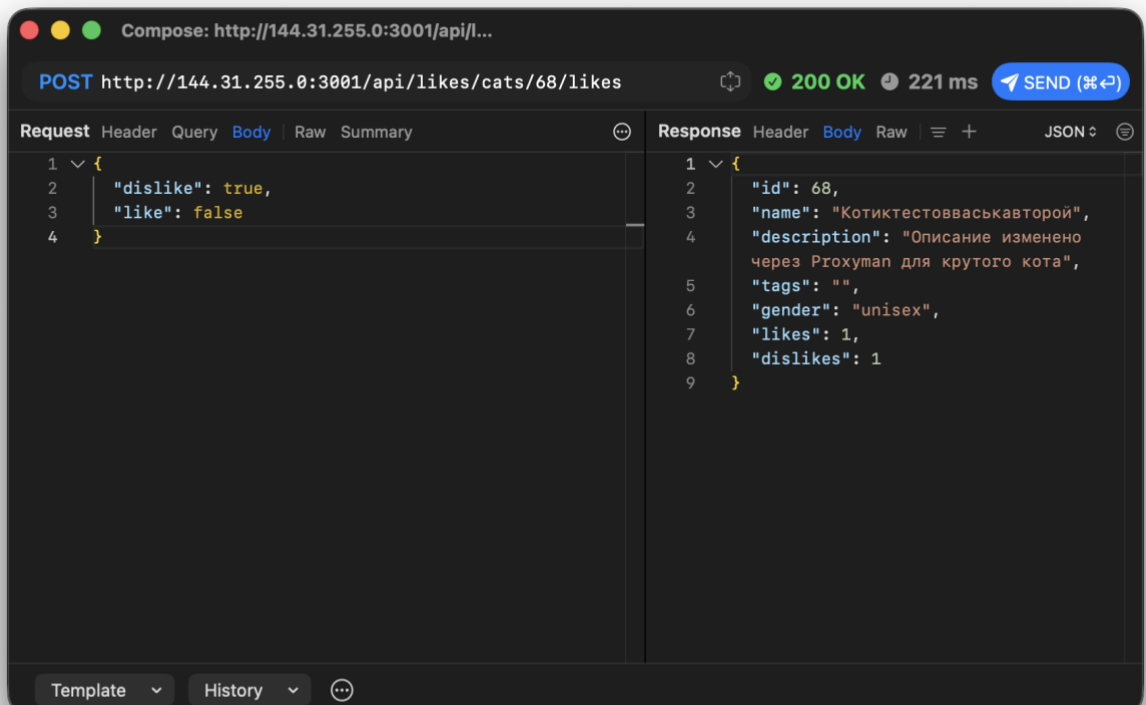


Рисунок 29 – Запрос дизлайка /api/likes/cats/68/likes

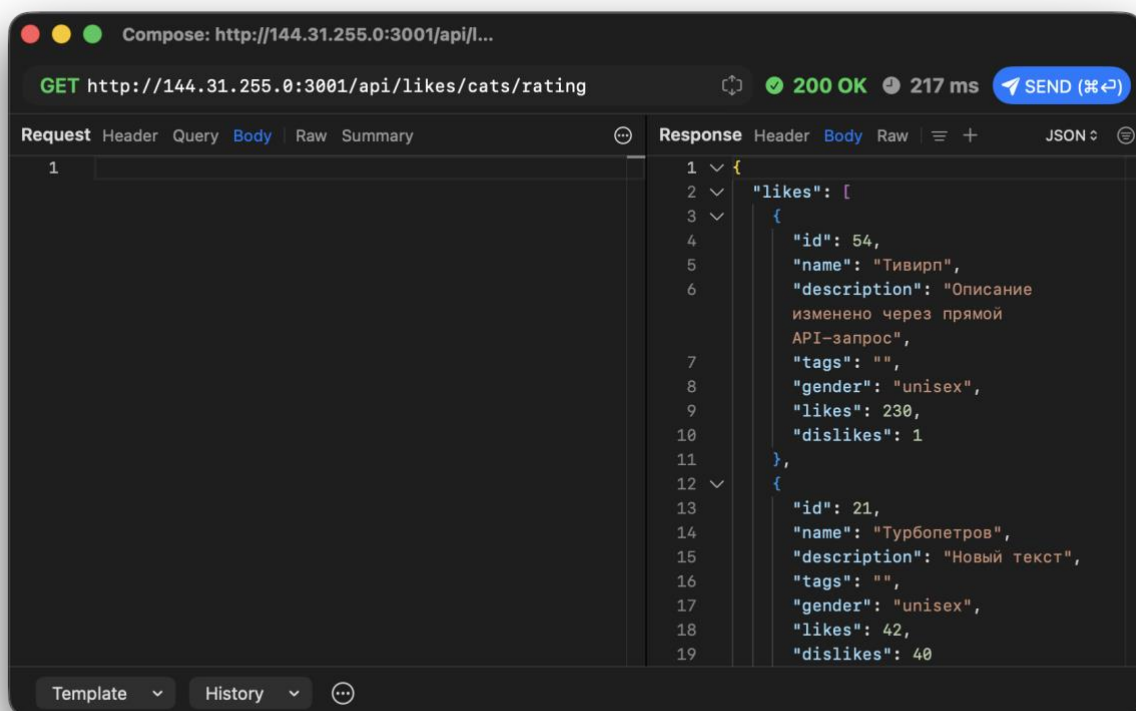


Рисунок 30 – запрос рейтинга /api/likes/cats/rating

3.3 Наблюдения по действиям приложения

1. Все основные действия приложения выполняются через mobile API с префиксом /api/.
2. Для открытия карточки используется прямой идентификатор id, например id=67.
3. Для лайка и дизлайка используется один endpoint /api/likes/cats/{catId}/likes, а действие определяется JSON-полями like и dislike.
4. После загрузки изображения сервер возвращает относительный путь /photos/..., который затем используется для просмотра изображения.
5. Рейтинг запрашивается отдельным `GET`-запросом и содержит сразу два списка: likes и dislikes.

3.5 Вывод по этапу

На этапе построения карты действий было подтверждено, какие HTTP-запросы реально отправляет Android-приложение Meowle. Полученная таблица показывает связь между действиями пользователя и backend endpoint-ами. Эти данные используются далее для изменения запросов через replay и проверки security-проблем.

4. ИЗМЕНЕНИЕ ЗАПРОСОВ

4.1 Цель этапа

Цель этапа - проверить, как backend API реагирует на измененные HTTP-запросы. Запросы изменяются через Proxymap. В качестве основной тестовой записи используется котик:

Таблица 7 – Тестовая запись

Поле	Значение
catId	68
name	Котиктестовваськавторой
description	Описание изменено через Proxymap
gender	unisex

Перед выполнением экспериментов важно фиксировать исходный запрос, измененный запрос, ответ сервера и изменение состояния данных.

4.2. Эксперимент 1. Изменение catId в URL карточки

Исходный запрос GET /api/core/cats/get-by-id?id=68

Body отсутствует.

Измененный запрос GET /api/core/cats/get-by-id?id=999999

Ожидаемый ответ сервера:

```
{
  "data": null,
  "isBoom": true,
  "isServer": false,
  "output": {
    "statusCode": 404,
    "payload": {
      "statusCode": 404,
      "error": "Not Found",
      "message": "cat not found"
    },
    "headers": {}
  }
}
```

Таблица 8 – Результат

Пункт	Значение
Изменилось ли состояние данных	Нет
Баг или security-проблема	Не баг, сервер корректно вернул 404
Как исправить	Дополнительно можно унифицировать формат ошибок и не раскрывать лишние внутренние поля isBoom, output

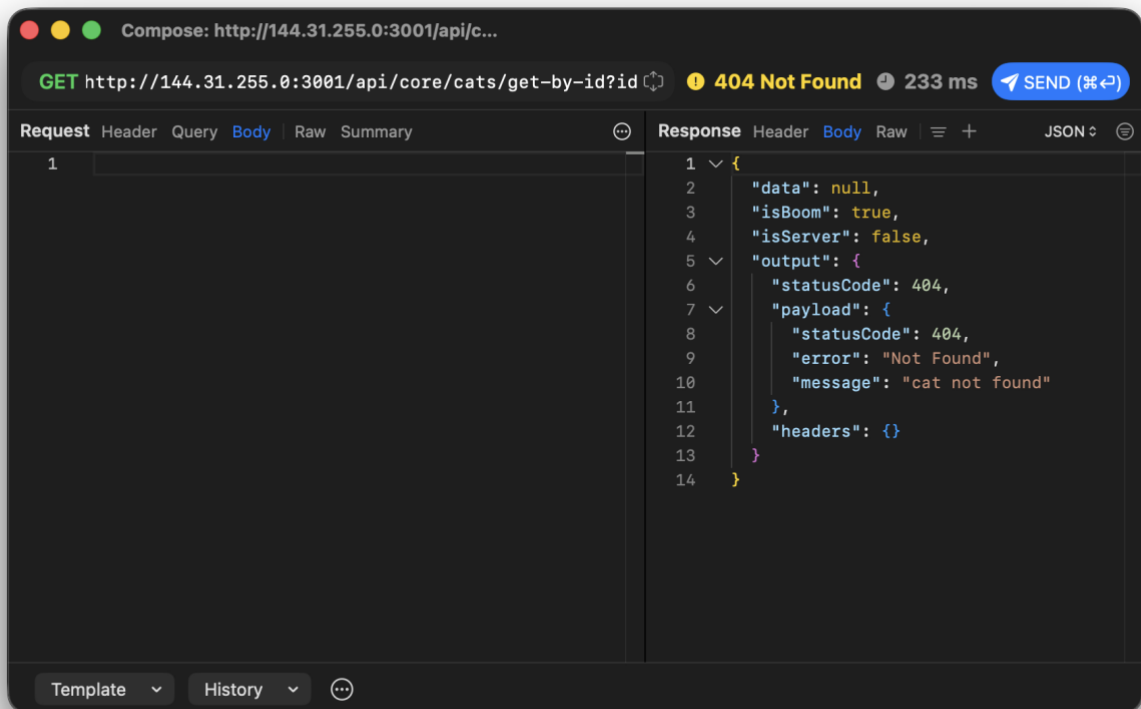


Рисунок 31 – Измененный catId=999999 в запросе карточки

4.3 Эксперимент 2. Повтор одного и того же лайка

Исходный запрос POST /api/likes/cats/68/likes с Content-Type: application/json; charset=UTF-8

```
{
  "dislike": false,
  "like": true
}
```

Измененный запрос

Изменений в запросе нет. Один и тот же запрос повторяется несколько раз через Replay.

POST /api/likes/cats/68/likes

```
{
  "dislike": false,
  "like": true
}
```

Ожидаемый ответ сервера

После каждого повтора счетчик 'likes' может увеличиваться:

```
{
  "id": 68,
  "name": "Котиктестовваськавторой",
  "description": "Описание изменено через Proxyma",
  "tags": "",
  "gender": "unisex",
  "likes": 3,
  "dislikes": 1
}
```

Таблица 9 – Результат

Пункт	Значение
Изменилось ли состояние данных	Да, счетчик лайков увеличивается
Баг или security-проблема	Security-проблема / бизнес-логическая ошибка
Почему это проблема	Один пользователь или клиент может повторять запрос и искусственно увеличивать рейтинг
Как исправить	Делать голосование идиempотентным: хранить голос пользователя и не увеличивать счетчик повторно

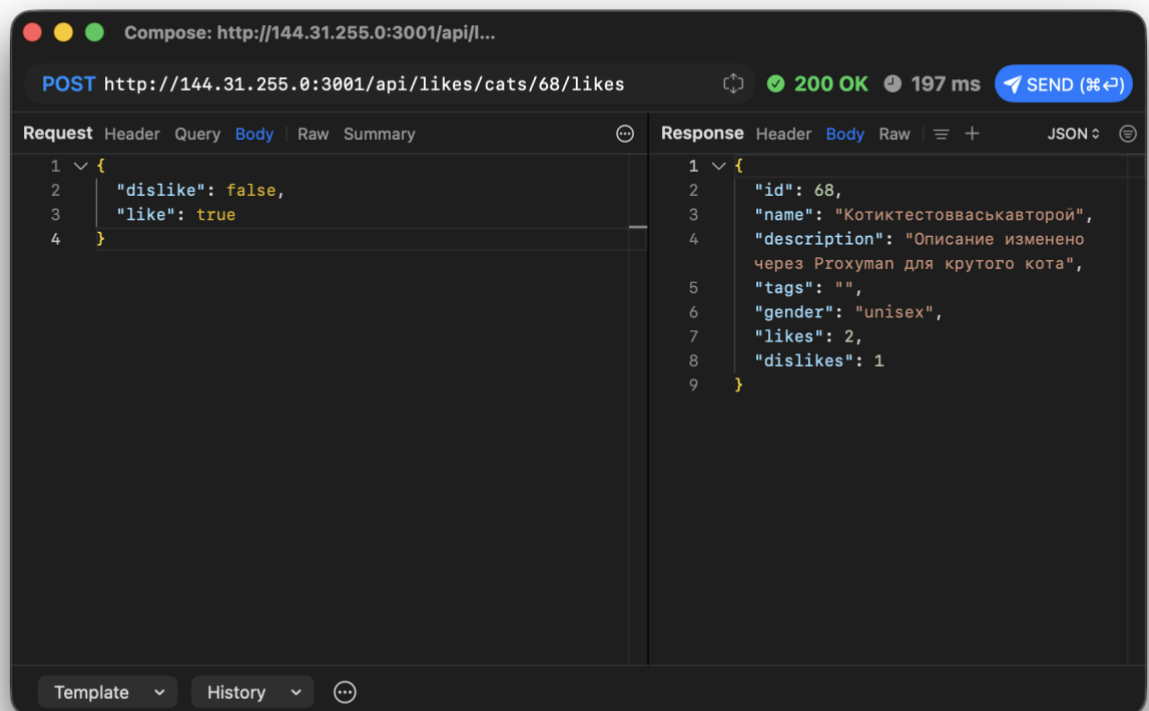


Рисунок 32 – Лайк

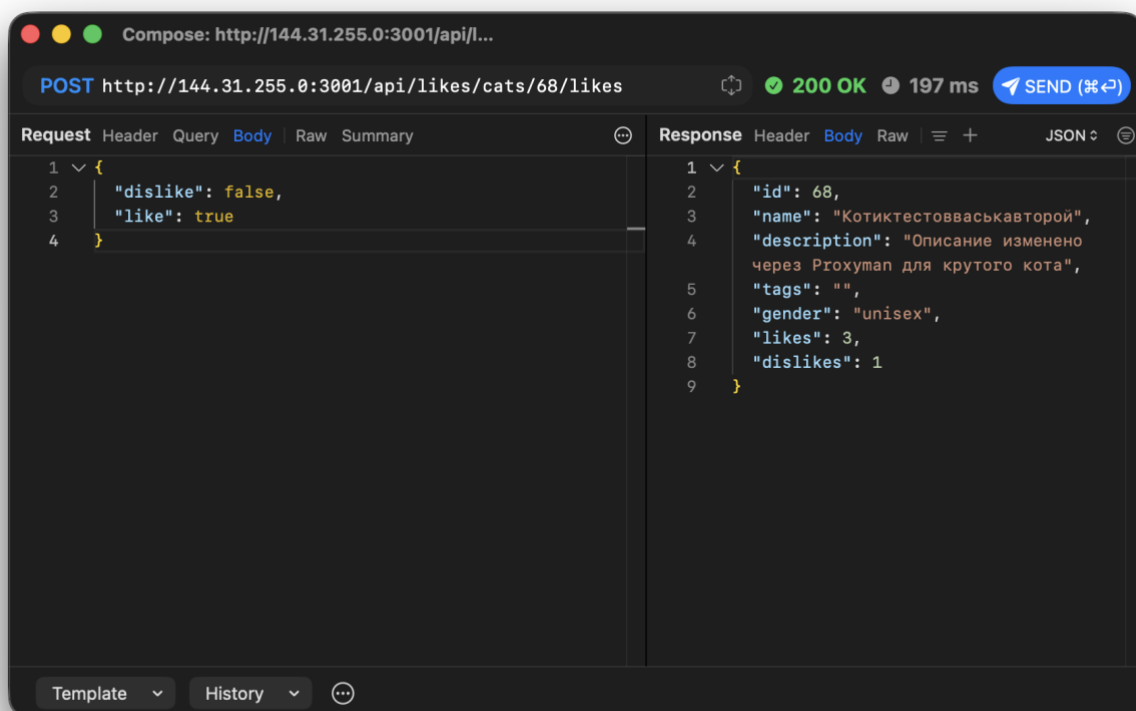


Рисунок 33 – Повторный лайк с увеличившимся счетчиком

4.4 Эксперимент 3. Изменение описания чужой записи

Для проверки используется запись, созданная не текущим пользователем. В предыдущей проверке использовалась публичная запись.

Таблица 10 – Публичная запись

Поле	Значение
catId	59
name	Тестик
Исходное описание	Публичный тестик

Исходный запрос POST /api/core/cats/save-description с Content-Type: application/json; charset=UTF-8

```
{
  "catId": 68,
  "catDescription": "Описание изменено через Proxuman"
}
```

Измененный запрос POST /api/core/cats/save-description с Content-Type: application/json; charset=UTF-8

```
{
  "catId": 59,
  "catDescription": "Временная проверка доступа"
}
```

Ожидаемый ответ сервера

```
{
  "id": 59,
  "name": "Тестик",
  "description": "Временная проверка доступа",
  "tags": "",

```

```

    "gender": "unisex",
    "likes": 0,
    "dislikes": 0
  }

```

После фиксации результата описание нужно восстановить:

```

  {
    "catId": 59,
    "catDescription": "Публичный тестик"
  }

```

Таблица 11 – Результат

Пункт	Значение
Изменилось ли состояние данных	Да, описание чужой записи изменилось
Баг или security-проблема	Security-проблема, IDOR / отсутствие проверки прав
Почему это проблема	Клиент может изменить объект, который он не создавал
Как исправить	Добавить аутентификацию, привязку объекта к владельцу и проверку прав на сервере

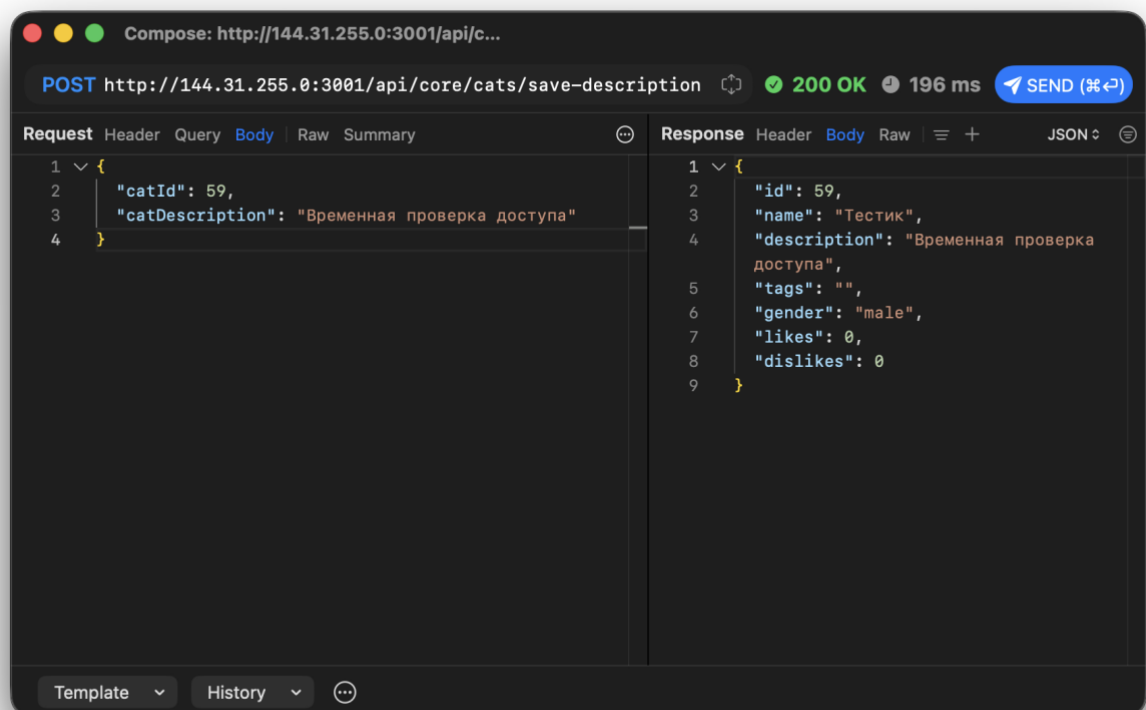


Рисунок 34 – Изменение описания чужой записи

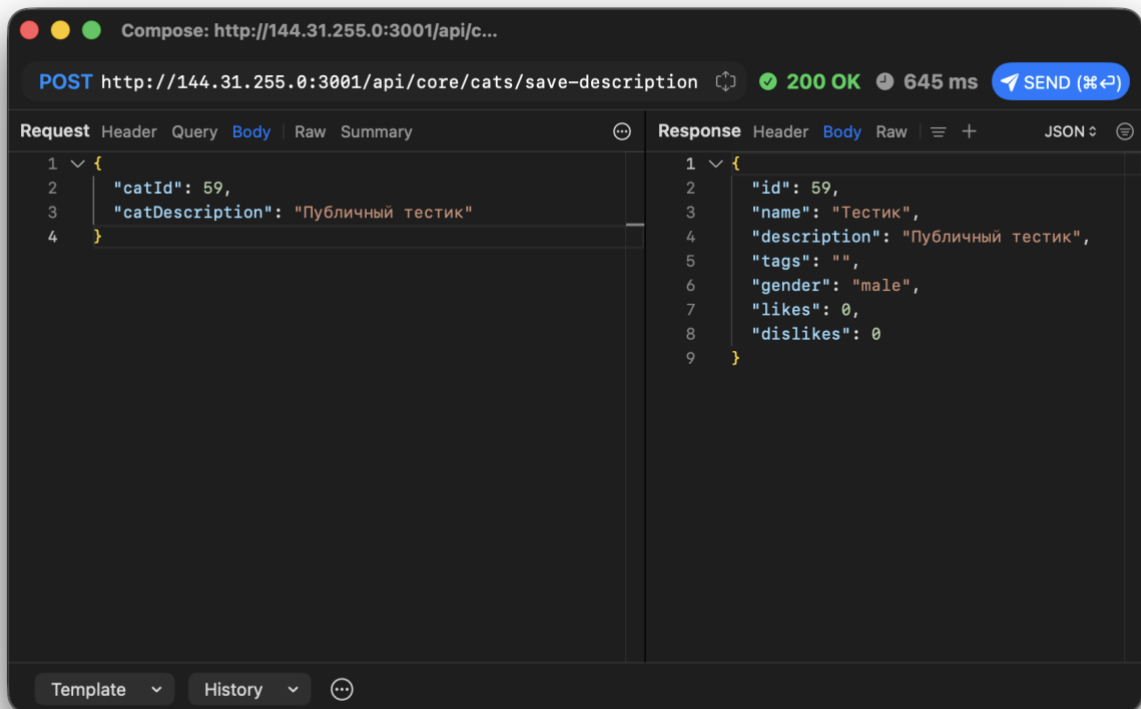


Рисунок 35 – Восстановление исходного описания

4.5 Эксперимент 4. Добавление дубликата имени

Исходный запрос POST /api/core/cats/add с Content-Type: application/json; charset=UTF-

8

```
{
  "cats": [
    {
      "name": "КотикТестовВаськаВторой",
      "gender": "unisex",
      "description": "КотикТестовВаськаВторой"
    }
  ]
}
```

Измененный запрос

Повторяется добавление того же имени:

```
{
  "cats": [
    {
      "name": "КотикТестовВаськаВторой",
      "gender": "unisex",
      "description": "Дубликат имени"
    }
  ]
}
```

Ожидаемый ответ сервера

```
{
  "cats": [
    {
```

```

    "errorDescription": "Такое значение уже существует"
  }
}
]
}

```

HTTP-статус при этом может быть 200 OK.

Таблица 12 – Результат

Пункт	Значение
Изменилось ли состояние данных	Нет, новая запись не создана
Баг или security-проблема	Баг обработки ошибок
Почему это проблема	Бизнес-ошибка возвращается как 200 OK, клиенту сложнее отличить успех от ошибки
Как исправить	Возвращать 409 Conflict для дубликата имени

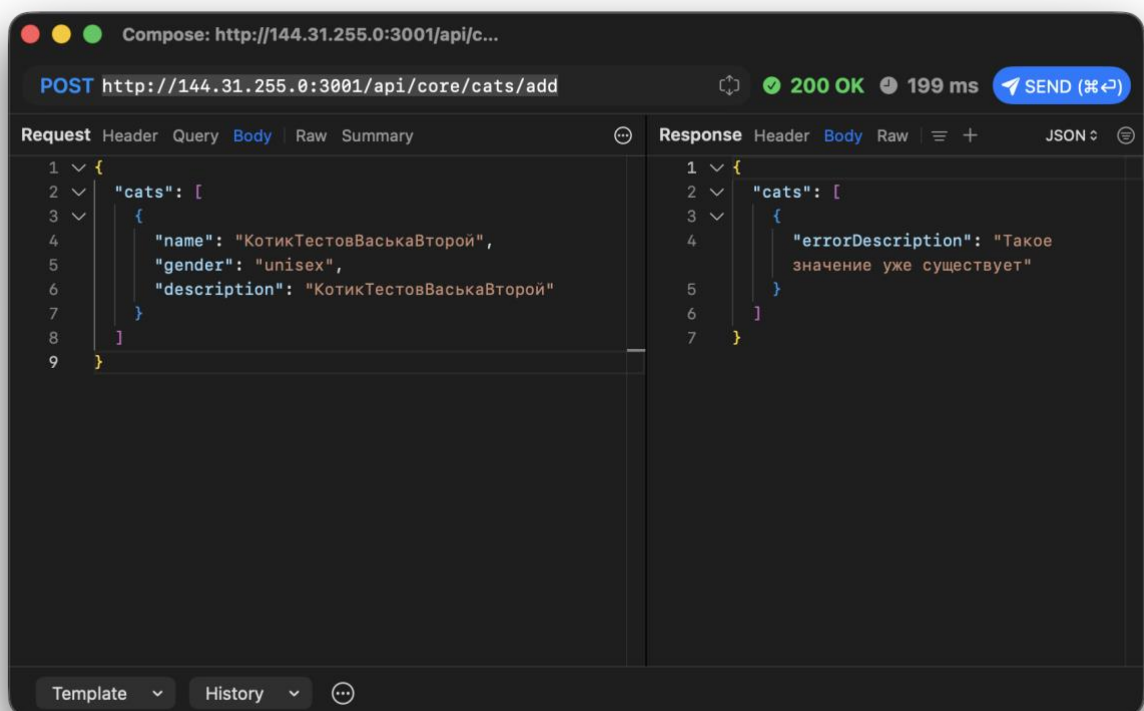


Рисунок 36 – Попытка добавить дубликат имени

4.6 Эксперимент 5. Невалидное значение gender

Исходный запрос

```

{
  "cats": [
    {
      "name": "Проверка Пол",
      "gender": "unisex",
      "description": "Корректный gender"
    }
  ]
}

```

Измененный запрос `POST /api/core/cats/add` с `Content-Type: application/json; charset=UTF-8`

```

{
  "cats": [
    {
      "name": "Проверка Пол",
      "gender": "dragon",
      "description": "Невалидный gender"
    }
  ]
}

```

Ожидаемый ответ сервера

В предыдущей проверке сервер не отклонял неизвестный `gender`, а создавал запись с `gender="unisex"`:

```

{
  "cats": [
    {
      "id": 0,
      "name": "Проверка пол",
      "description": "Невалидный gender",
      "tags": "",
      "gender": "unisex",
      "likes": 0,
      "dislikes": 0
    }
  ]
}

```

id нужно заменить на фактический из ответа.

Таблица 13 – Результат

Пункт	Значение
Изменилось ли состояние данных	Да, может создаться запись
Баг или security-проблема	Баг валидации входных данных
Почему это проблема	Сервер молча заменяет недопустимое значение вместо отказа
Как исправить	Валидировать enum gender и возвращать 400 Bad Request

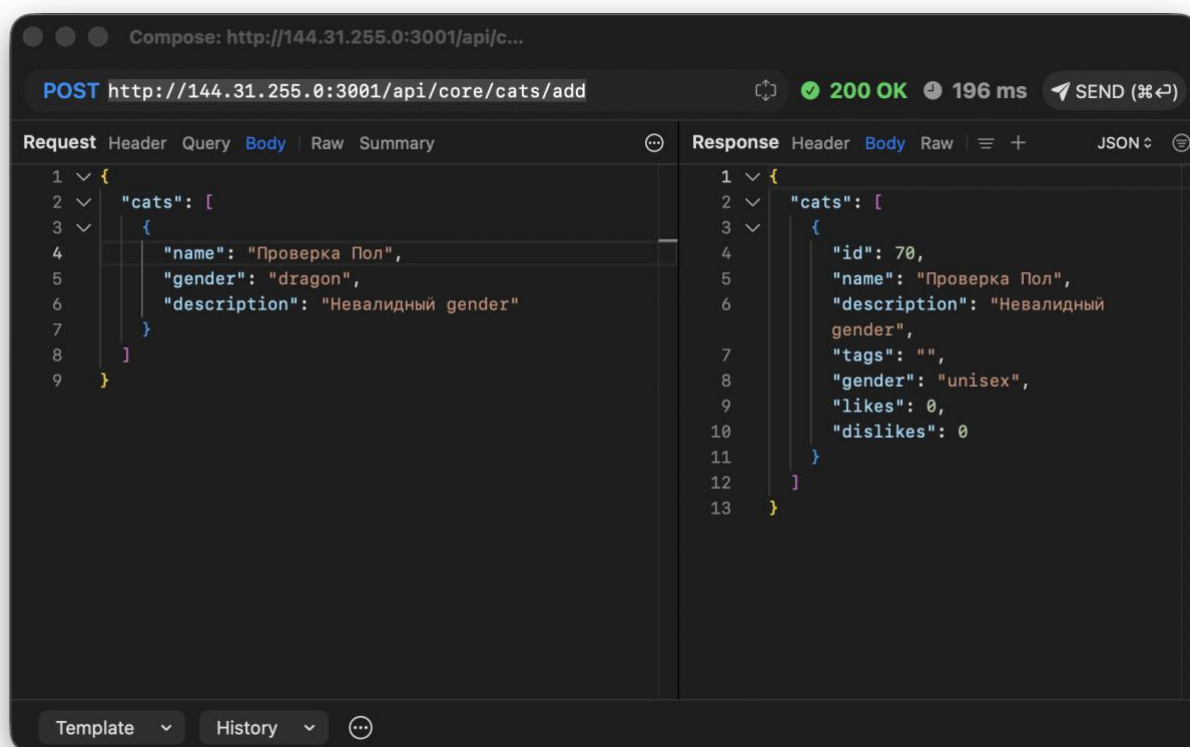


Рисунок 37 – Невалидный gender

4.7 Эксперимент 6. Невалидное значение name

Измененный запрос с латиницей POST `/api/core/cats/add` с Content-Type: application/json; charset=UTF-8

```
{
  "cats": [
    {
      "name": "InvalidName",
      "gender": "unisex",
      "description": "Латиница в имени"
    }
  ]
}
```

Ожидаемый ответ сервера

```
{
  "cats": [
    {
      "errorDescription": "Только имена на русском!"
    }
  ]
}
```

Измененный запрос с цифрами

```
{
  "cats": [
    {
      "name": "Имя123",
      "gender": "unisex",
      "description": "Цифры в имени"
    }
  ]
}
```

```

    ]
  }
  Ожидаемый ответ сервера
  {
    "cats": [
      {
        "errorDescription": "Цифры не принимаются!"
      }
    ]
  }
}

```

HTTP-статус при ошибке может быть 200 OK.

Таблица 14 – Результат

Пункт	Значение
Изменилось ли состояние данных	Нет
Баг или security-проблема	Баг HTTP-статуса
Почему это проблема	Валидация есть, но ошибка возвращается как успешный HTTP-ответ
Как исправить	Возвращать 400 Bad Request с ошибкой валидации

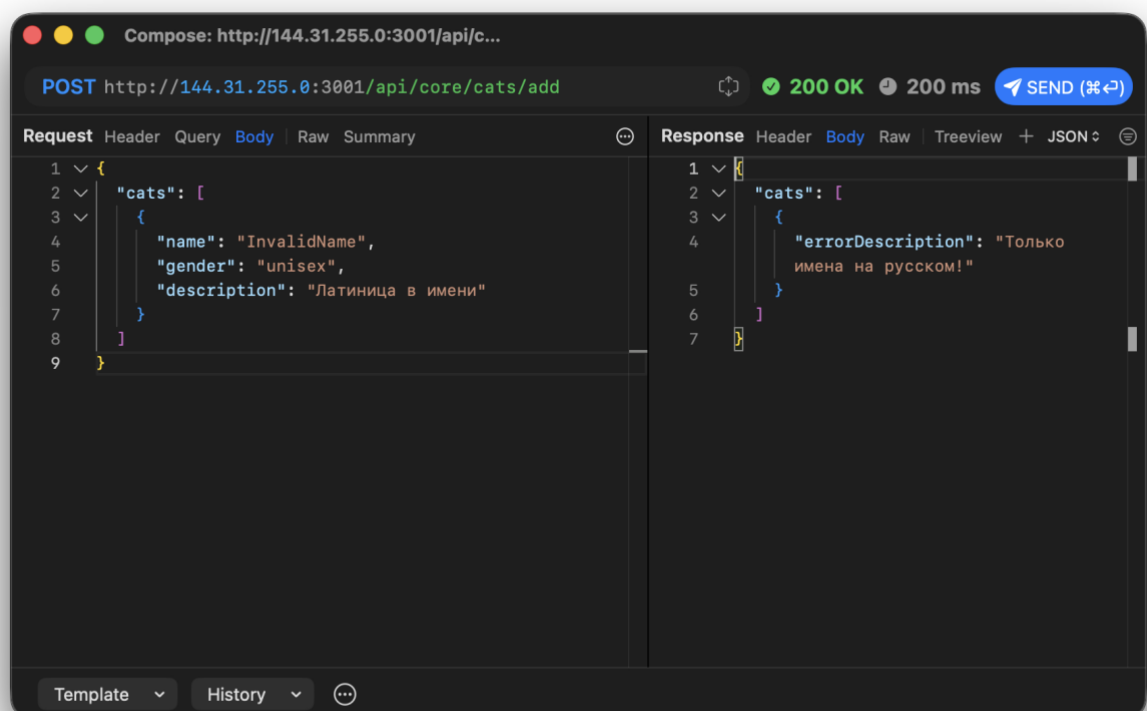


Рисунок 38 – Невалидное имя латиницей

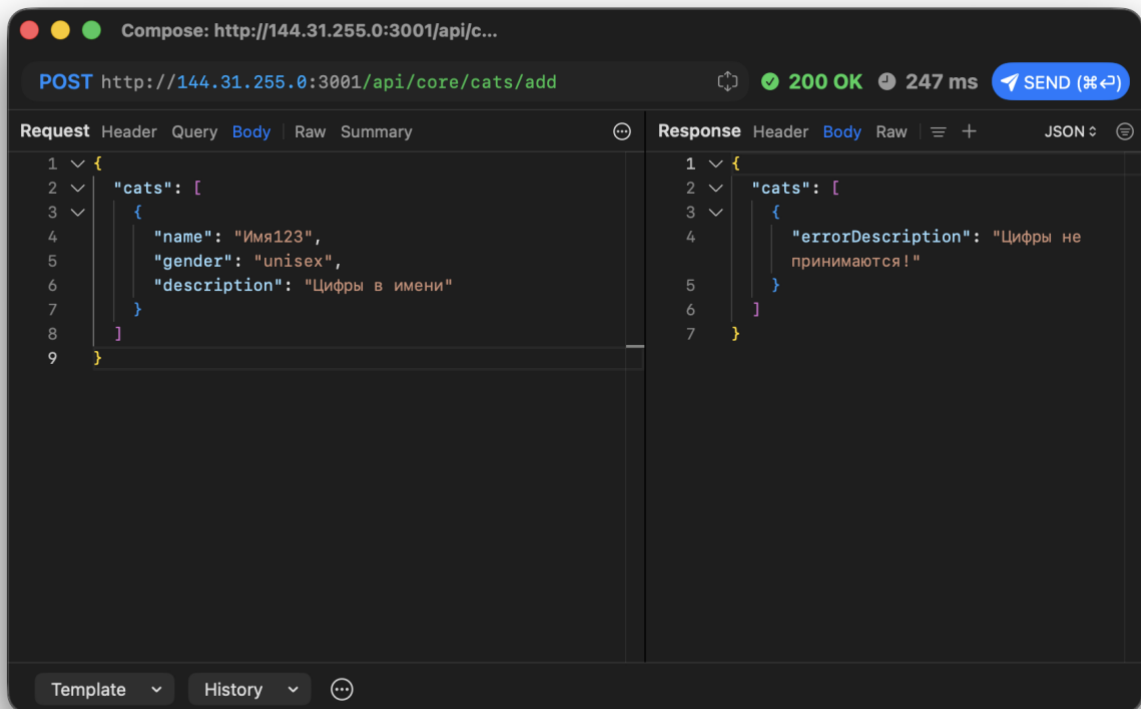


Рисунок 39 – Невалидное имя с цифрами

4.8 Эксперимент 7. Невалидное значение `catDescription`

Исходный запрос

```
{
  "catId": 68,
  "catDescription": "Описание изменено через Proxuman"
}
```

Измененный запрос: число вместо строки POST /api/core/cats/save-description с Content-Type: application/json; charset=UTF-8

```
{
  "catId": 68,
  "catDescription": 12345
}
```

Ожидаемый ответ сервера

Сервер может принять значение и сохранить его как строку:

```
{
  "id": 68,
  "name": "Котиктестовваськавторой",
  "description": "12345",
  "tags": "",
  "gender": "unisex",
  "likes": 0,
  "dislikes": 0
}
```

Измененный запрос: пустая строка

```
{
  "catId": 68,
  "catDescription": ""
}
```

}

Таблица 15 – Результат

Пункт	Значение
Изменилось ли состояние данных	Да, описание меняется
Баг или security-проблема	Баг валидации
Почему это проблема	Сервер не проверяет тип и содержимое описания достаточно строго
Как исправить	Проверить JSON schema: catDescription должен быть строкой допустимой длины и формата

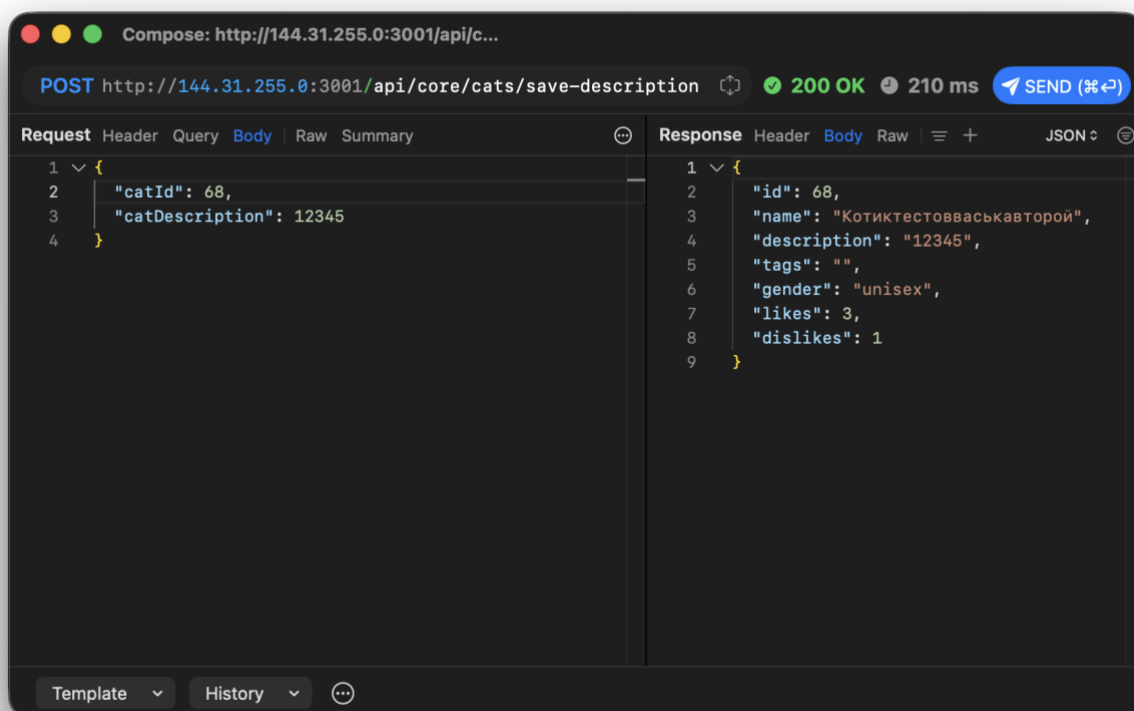


Рисунок 40 – числовой catDescription

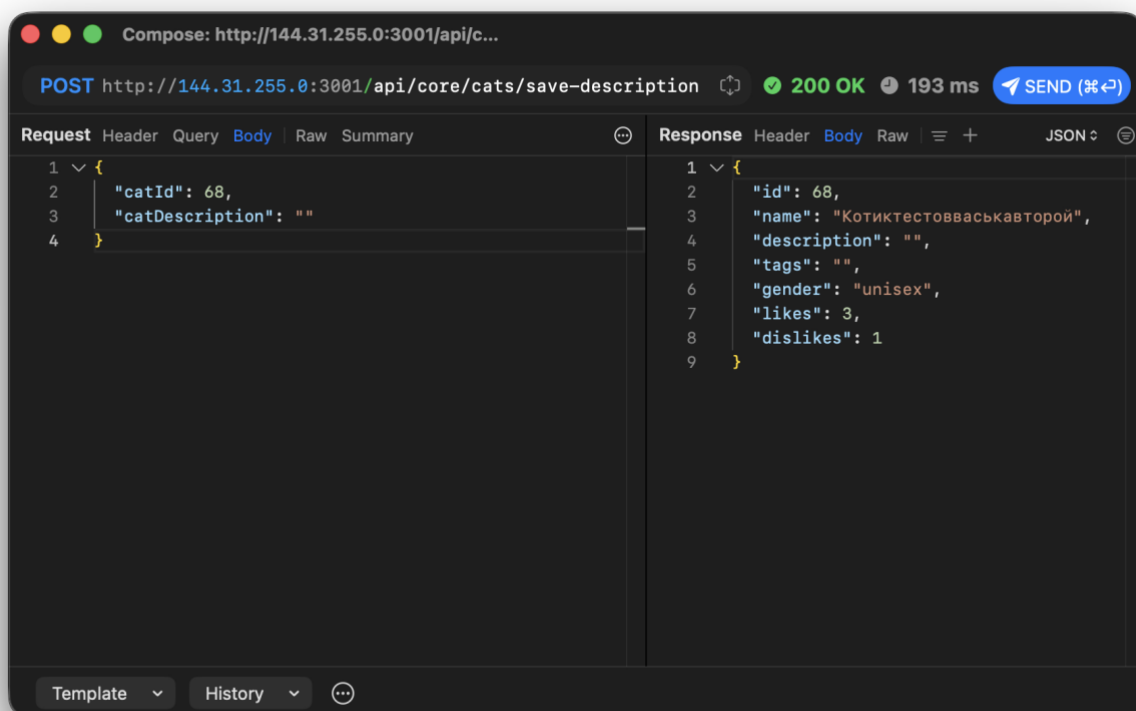


Рисунок 41 – Пустой catDescription

4.9 Эксперимент 8. Вызов web API-методов вне мобильного сценария

Android-приложение использует mobile API с префиксом api/, но Swagger также показывает web API без этого префикса. Проверим, что web API можно вызвать напрямую.

Исходный мобильный запрос POST /api/likes/cats/68/likes

```
{
  "dislike": false,
  "like": true
}
```

Измененный web API запрос POST /cats/68/like

Body отсутствует.

Дополнительный web API запрос POST /cats/68/dislike

Body отсутствует.

Ожидаемый ответ сервера

Ответ 200 OK, а состояние счетчиков изменится.

Таблица 16 – Результат

Пункт	Значение
Изменилось ли состояние данных	Да, счетчики могут измениться
Баг или security-проблема	Потенциальная security-проблема
Почему это проблема	Методы, которых нет в мобильном UI, доступны напрямую
Как исправить	Применять одинаковую авторизацию, rate limit и бизнес-правила ко всем endpoint-ам

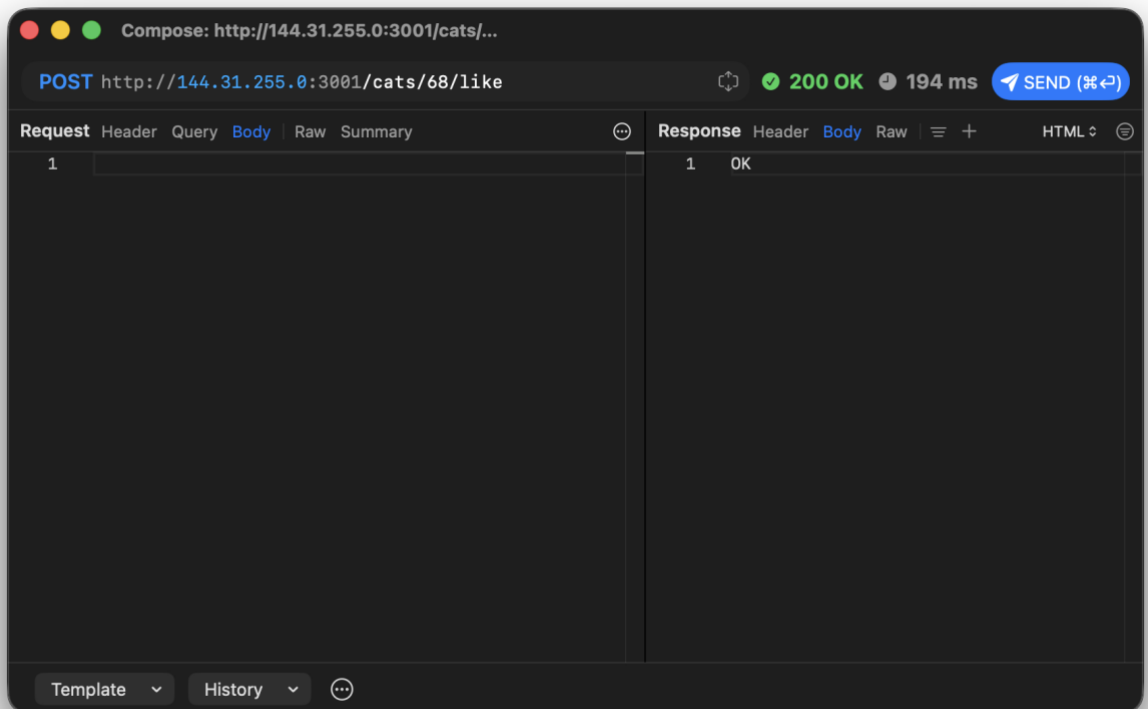


Рисунок 42 – Вызов POST /cats/68/like

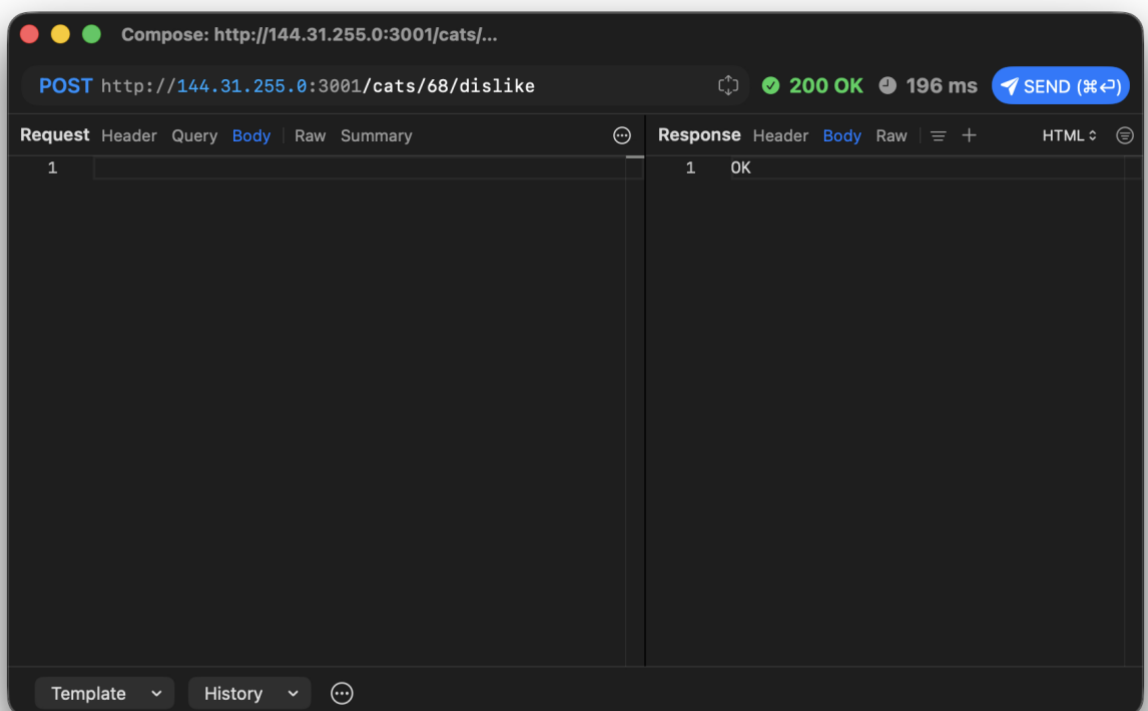


Рисунок 43 – Вызов POST /cats/68/dislike

4.10 Эксперимент 9. Обращение к несуществующему `catId`
Измененный запрос POST /api/likes/cats/999999/likes с Content-Type: application/json;
charset=UTF-8

```

{
  "dislike": false,
  "like": true
}
Ожидаемый ответ сервера
{
  "data": null,
  "isBoom": true,
  "isServer": false,
  "output": {
    "statusCode": 404,
    "payload": {
      "statusCode": 404,
      "error": "Not Found",
      "message": "cat not found"
    },
    "headers": {}
  }
}

```

Таблица 17 – Результат

Пункт	Значение
Изменилось ли состояние данных	Нет
Баг или security-проблема	Не security-проблема, но формат ошибки можно улучшить
Как исправить	Возвращать единый JSON-формат ошибки без внутренних служебных полей

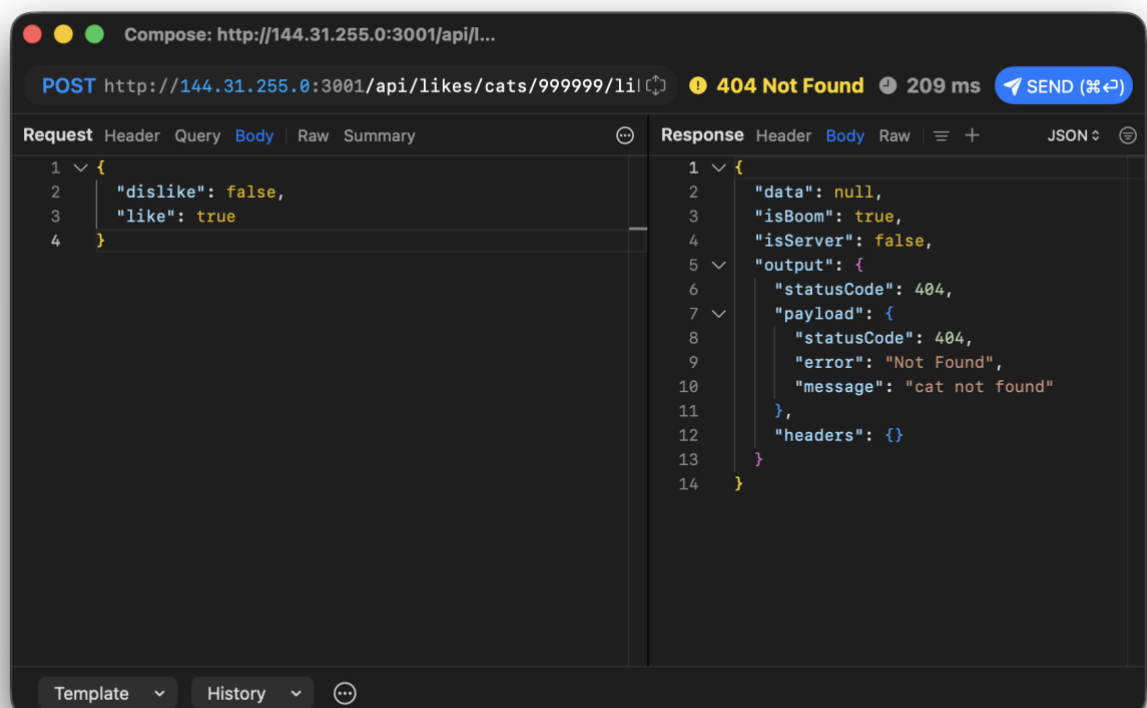


Рисунок 44 – Лайк несуществующего catId

4.11 Эксперимент 10. Загрузка файла

Корректное изображение POST /api/photos/cats/68/upload
multipart:

field name: file

filename: котяра.jpeg

Content-Type: image/jpeg

file: котяра.jpeg

Ожидаемый ответ:

```
{
  "fileUrl": "/photos/image-....jpeg"
}
```

Файл с неправильным MIME type

Тот же файл можно отправить с неправильным MIME:

field name: file

filename: котяра.jpeg

Content-Type: text/plain

Файл с неправильным расширением

field name: file

filename: котяра.txt

Content-Type: image/jpeg

Ожидаемое поведение

Корректный файл должен загрузиться. Файл с неправильным MIME или расширением должен быть отклонен с 400 Bad Request или 415 Unsupported Media Type.

Если сервер возвращает 500 Internal Server Error, это баг обработки файла.

Таблица 18 – Результат

Пункт	Значение
Изменилось ли состояние данных	Для корректного файла да, фото добавлено
Баг или security-проблема	При 500 - баг обработки ошибок и валидации файла
Как исправить	Проверять MIME, расширение и сигнатуру файла до сохранения; возвращать 4xx вместо 500

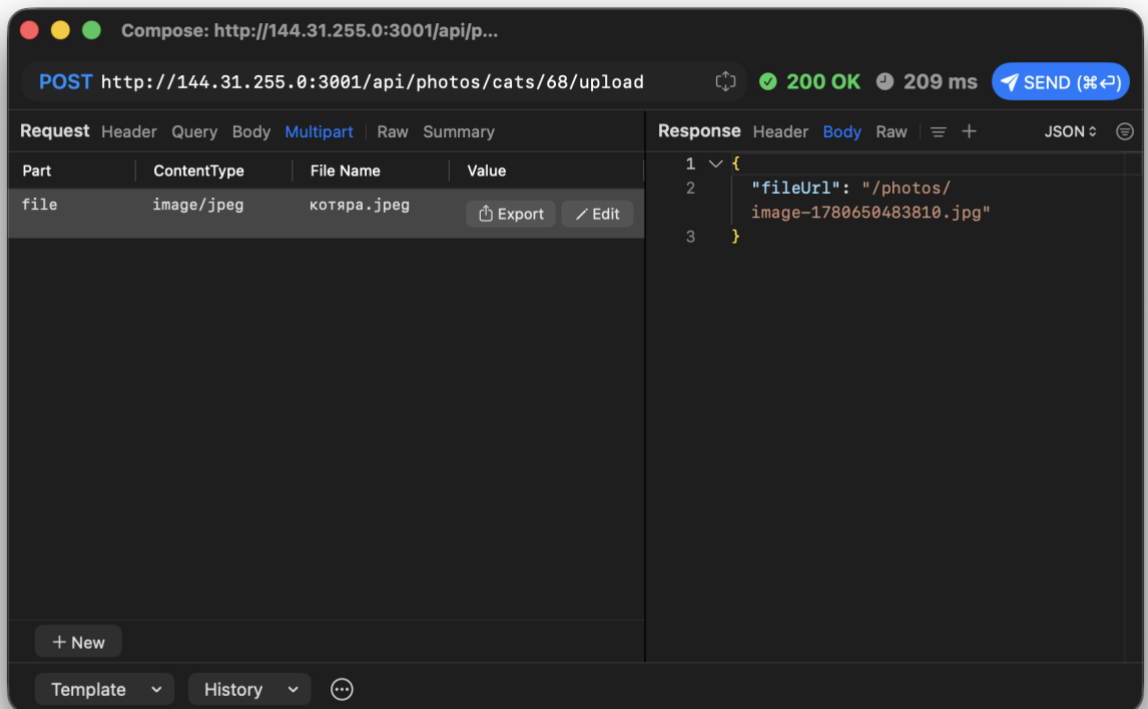


Рисунок 45 - Успешная загрузка изображения

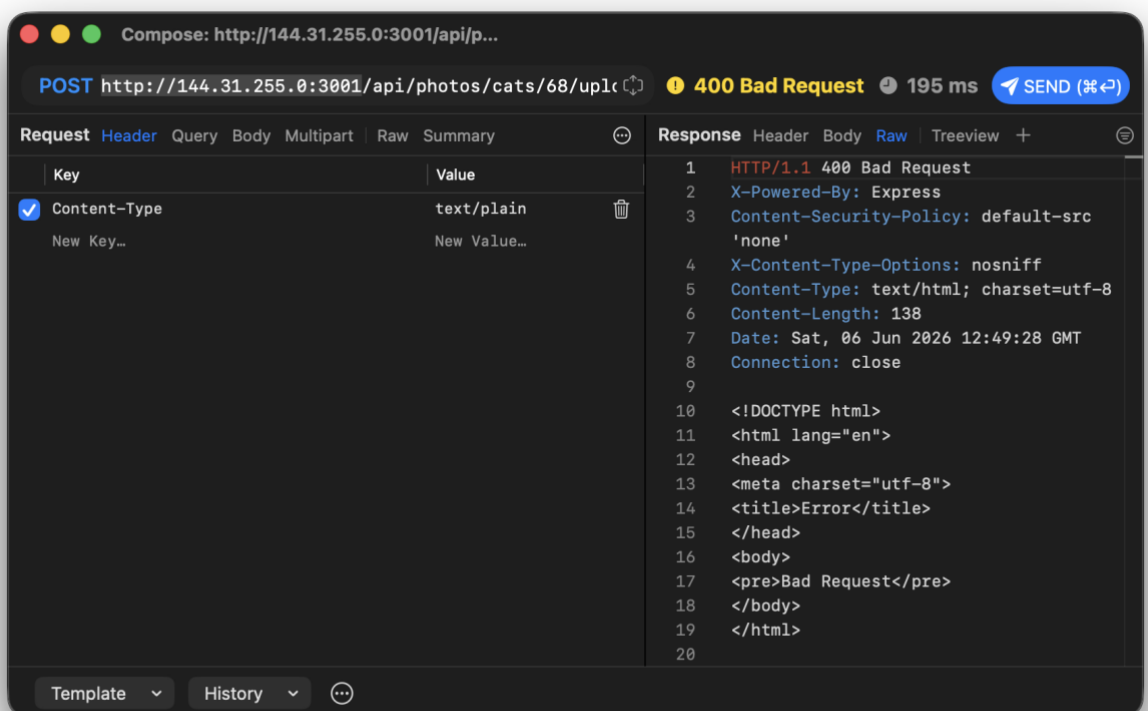


Рисунок 46 – upload с неправильным MIME type

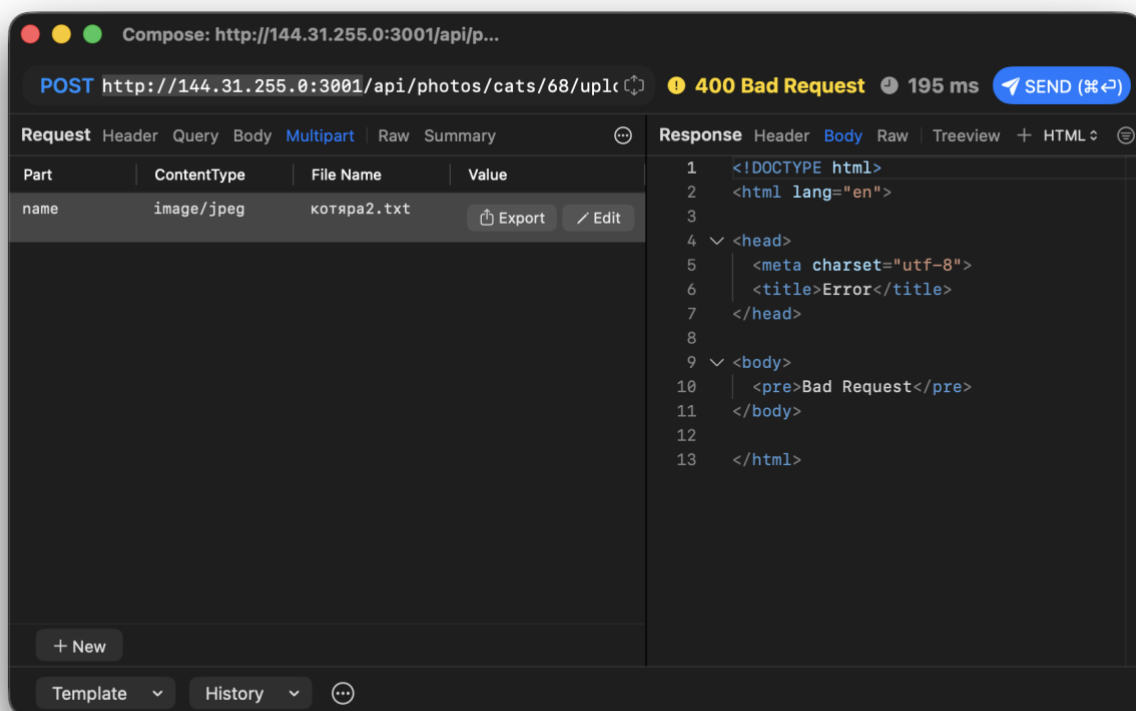


Рисунок 47 – upload с неправильным расширением

4.12 Итог экспериментов

Таблица 19 – Итоговая таблица

N	Эксперимент	Состояние данных изменилось	Классификация
1	Изменение catId в карточке	Нет	Корректная обработка 404
2	Повтор лайка	Да	Security / бизнес-логическая ошибка
3	Изменение чужого описания	Да	High security, IDOR
4	Дубликат имени	Нет	Баг HTTP-статуса
5	Невалидный gender	Да	Баг валидации
6	Невалидный name	Нет	Баг HTTP-статуса
7	Невалидный catDescription	Да	Баг валидации
8	Вызов web API вне мобильного сценария	Да	Потенциальная security-проблема
9	Несуществующий catId	Нет	Корректный 404
10	Некорректный upload	Зависит от ответа	Баг валидации файла при 500

4.13 Вывод по этапу

Изменение запросов показало, что backend API принимает запросы напрямую и не должен полагаться на ограничения мобильного интерфейса. Основные проблемы связаны с отсутствием проверки прав на изменение чужих данных, возможностью повторять действия напрямую через API, слабой валидацией отдельных параметров и некорректными HTTP-статусами для бизнес-ошибок.

5. ИЗМЕНЕНИЕ ОТВЕТОВ

5.1 Цель этапа

Цель этапа - проверить, как мобильное приложение реагирует на измененные ответы backend API. На этом этапе изменяется не запрос клиента, а ответ сервера, который проходит через Proxuman. Для подмены ответов использовался Proxuman. Основной способ проверки - включение breakpoint на response или настройка Map Local для конкретного URL. После подмены выполнялось то же действие в приложении, и сравнивалось отображение данных до и после изменения ответа.

5.2. Настройка подмены ответа в Proxuman

Таблица 20 – Два подхода

Способ	Когда удобно использовать
Breakpoint на Response	Для разовой ручной подмены JSON-ответа перед передачей в приложение
Map Local	Для стабильной подмены ответа на заранее подготовленный JSON-файл

Для выполнения экспериментов использовался следующий общий порядок:

1. Включить прокси и открыть нужное действие в мобильном приложении.
2. Найти запрос в Proxuman.
3. Включить response breakpoint или Map Local для нужного URL.
4. Повторить действие в приложении.
5. Изменить JSON-ответ.
6. Отправить измененный ответ дальше в приложение.
7. Сделать скриншот состояния до и после.
8. Отключить правило подмены и повторить запрос, чтобы убедиться, что серверные данные не изменились.

5.3. Эксперимент 1. Подмена имени и описания котика в ответе поиска

Исходное действие

В приложении выполняется поиск котика. Приложение отправляет запрос:

POST /api/core/cats/search

Content-Type: application/json; charset=UTF-8

Пример body:

```
{
  "gender": "unisex",
  "name": "КотикТестовВаськаВторой",
  "order": "asc"
}
```

Исходный ответ

```
{
  "groups": [
    {
      "title": "K",
      "cats": [
        {
          "id": 68,
          "name": "Котиктестовваськавторой",
          "description": "Описание изменено через Proxuman",
          "tags": "",
          "gender": "unisex",
          "likes": 0,

```

```

        "dislikes": 0
      }
    ],
    "count": 1
  }
]
}

```

Подмененный ответ

В response body через Proxuman изменяются поля name и description:

```

{
  "groups": [
    {
      "title": "T",
      "cats": [
        {
          "id": 68,
          "name": "ТестВебчик",
          "description": "Описание подменено только в ответе Proxuman",
          "tags": "",
          "gender": "unisex",
          "likes": 0,
          "dislikes": 0
        }
      ],
      "count": 1
    }
  ]
}

```

Таблица 21 – Результат

Пункт	Значение
Что изменилось в приложении	В результатах поиска отображается имя ТестВебчик и измененное описание
Изменилось ли состояние сервера	Нет
Как проверено	После отключения подмены и повторного поиска возвращается исходное имя и описание
Вывод	Клиент доверяет полученному JSON-ответу и отображает измененные данные

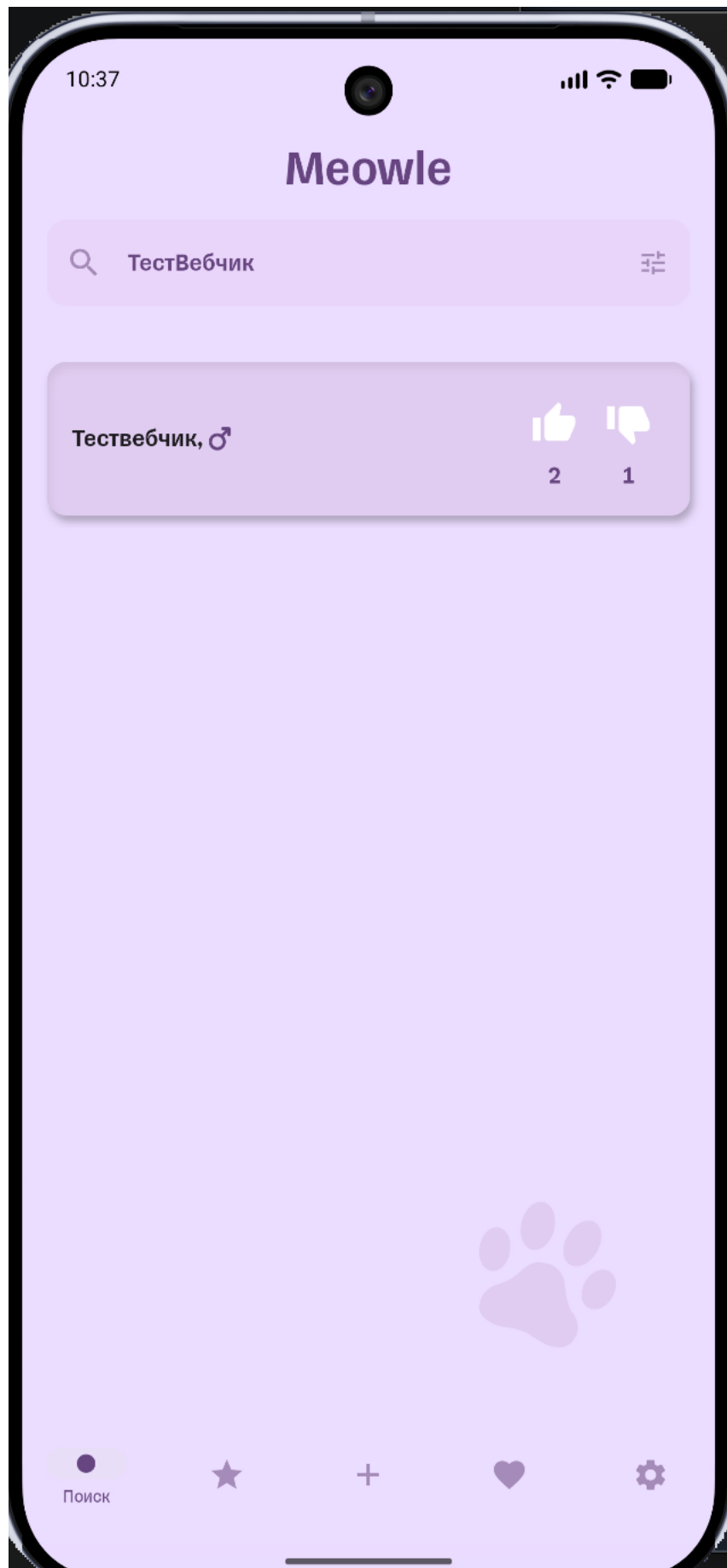


Рисунок 48 – Результат поиска до подмены ответа



Рисунок 49 – Результат поиска после подмены имени и описания

5.4 Эксперимент 2. Подмена количества лайков и дизлайков

Исходное действие

В приложении выполняется лайк или дизлайк котика. Приложение отправляет запрос:
POST /api/likes/cats/68/likes с Content-Type: application/json; charset=UTF-8

Пример body для лайка:

```
{  
  "dislike": false,  
  "like": true  
}
```

Исходный ответ

Фактические значения счетчиков зависят от состояния стенда.

Подмененный ответ

В response body через Proxuman изменяются поля likes и dislikes:

```
{  
  "id": 68,  
  "name": "Котиктестовваськавторой",  
  "description": "Описание изменено через Proxuman",  
  "tags": "",  
  "gender": "unisex",  
  "likes": 5000,  
  "dislikes": 5000  
}
```

Таблица 22 – Результат

Пункт	Значение
Что изменилось в приложении	Приложение отображает подмененные счетчики лайков и дизлайков
Изменилось ли состояние сервера	Нет
Как проверено	После отключения подмены и повторного открытия карточки возвращаются реальные счетчики
Вывод	Отображаемые счетчики на клиенте можно изменить через сетевой ответ, но это не меняет серверные данные

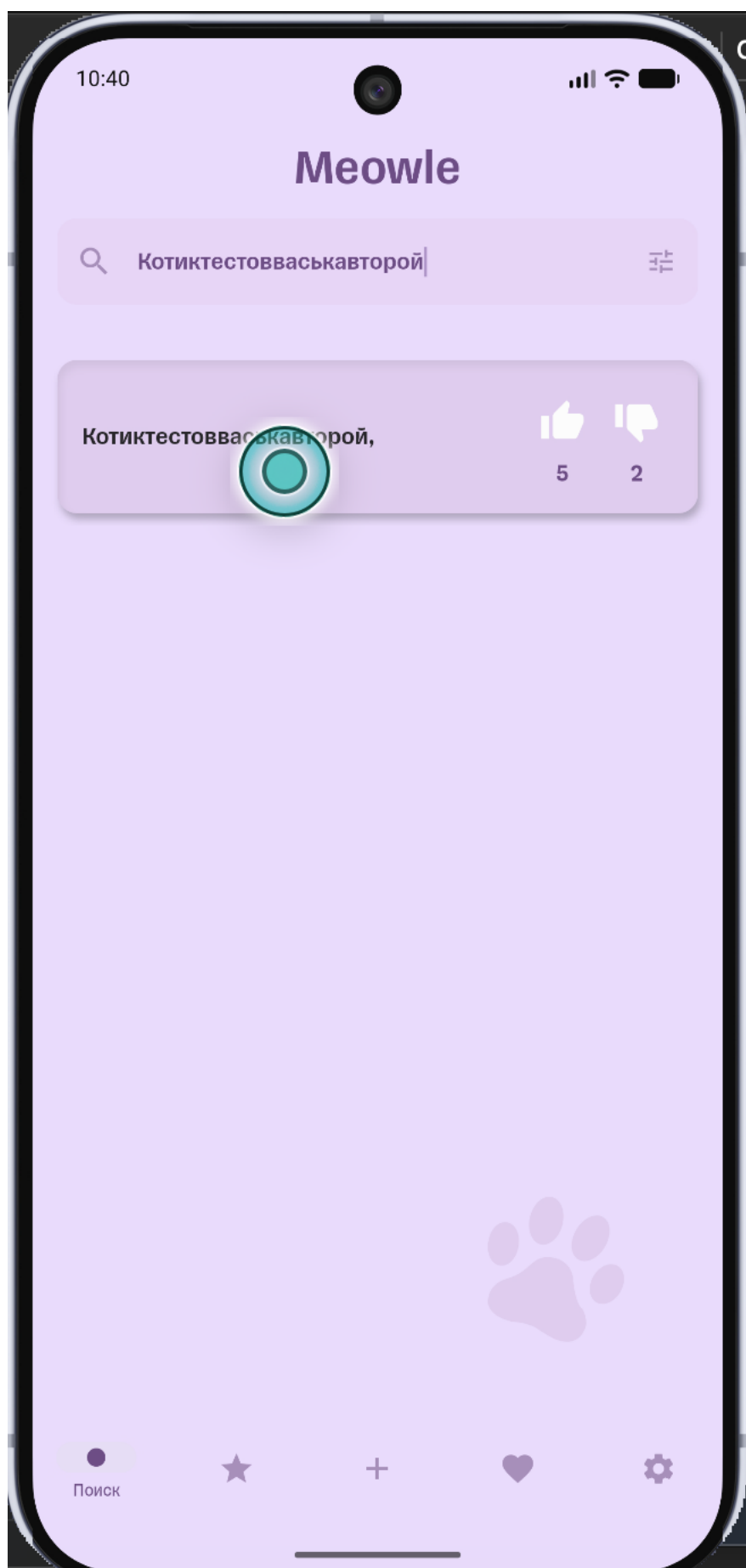


Рисунок 50 – Счетчики лайков и дизлайков до подмены

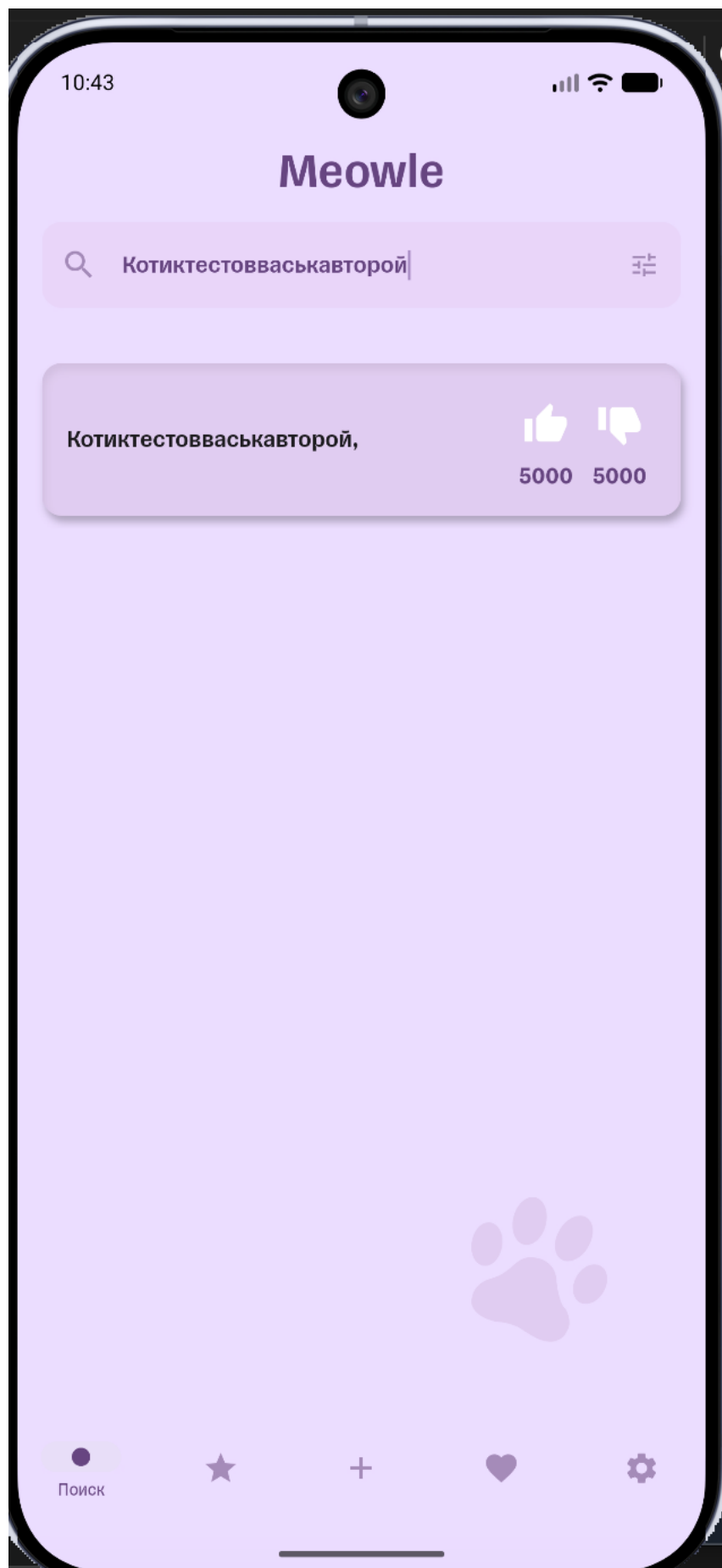


Рисунок 51 – Счетчики лайков и дизлайков после подмены

5.5 Эксперимент 3. Подмена списка результатов поиска

Исходное действие

В приложении выполняется поиск котиков через: POST /api/core/cats/search

Исходный ответ

Ответ содержит список найденных котиков, сгруппированный по первой букве:

```
{
  "groups": [
    {
      "title": "К",
      "cats": [
        {
          "id": 68,
          "name": "Котиктестовваськавторой",
          "description": "Описание изменено через Proxuman",
          "tags": "",
          "gender": "unisex",
          "likes": 5,
          "dislikes": 2
        }
      ],
      "count": 1
    }
  ]
}
```

Подмененный ответ

Через Proxuman возвращается другой список:

```
{
  "groups": [
    {
      "title": "Т",
      "cats": [
        {
          "id": 68,
          "name": "ТестВебчик",
          "description": "Первая подмененная запись",
          "tags": "",
          "gender": "unisex",
          "likes": 10,
          "dislikes": 0
        },
        {
          "id": 69,
          "name": "ТестовыйКот",
          "description": "Вторая подмененная запись",
          "tags": "",
          "gender": "male",
          "likes": 5,
          "dislikes": 1
        }
      ],
      {
        "id": 70,
```

```

    "name": "ТестоваяКошка",
    "description": "Третья подмененная запись",
    "tags": "",
    "gender": "female",
    "likes": 3,
    "dislikes": 2
  }
],
"count": 3
}
]
}

```

Таблица 22 – Результат

Пункт	Значение
Что изменилось в приложении	В списке поиска отображаются подмененные записи
Изменилось ли состояние сервера	Нет
Как проверено	После отключения подмены приложение снова получает реальный список с сервера
Вывод	Список на клиенте формируется на основе JSON-ответа и может быть визуально изменен прокси

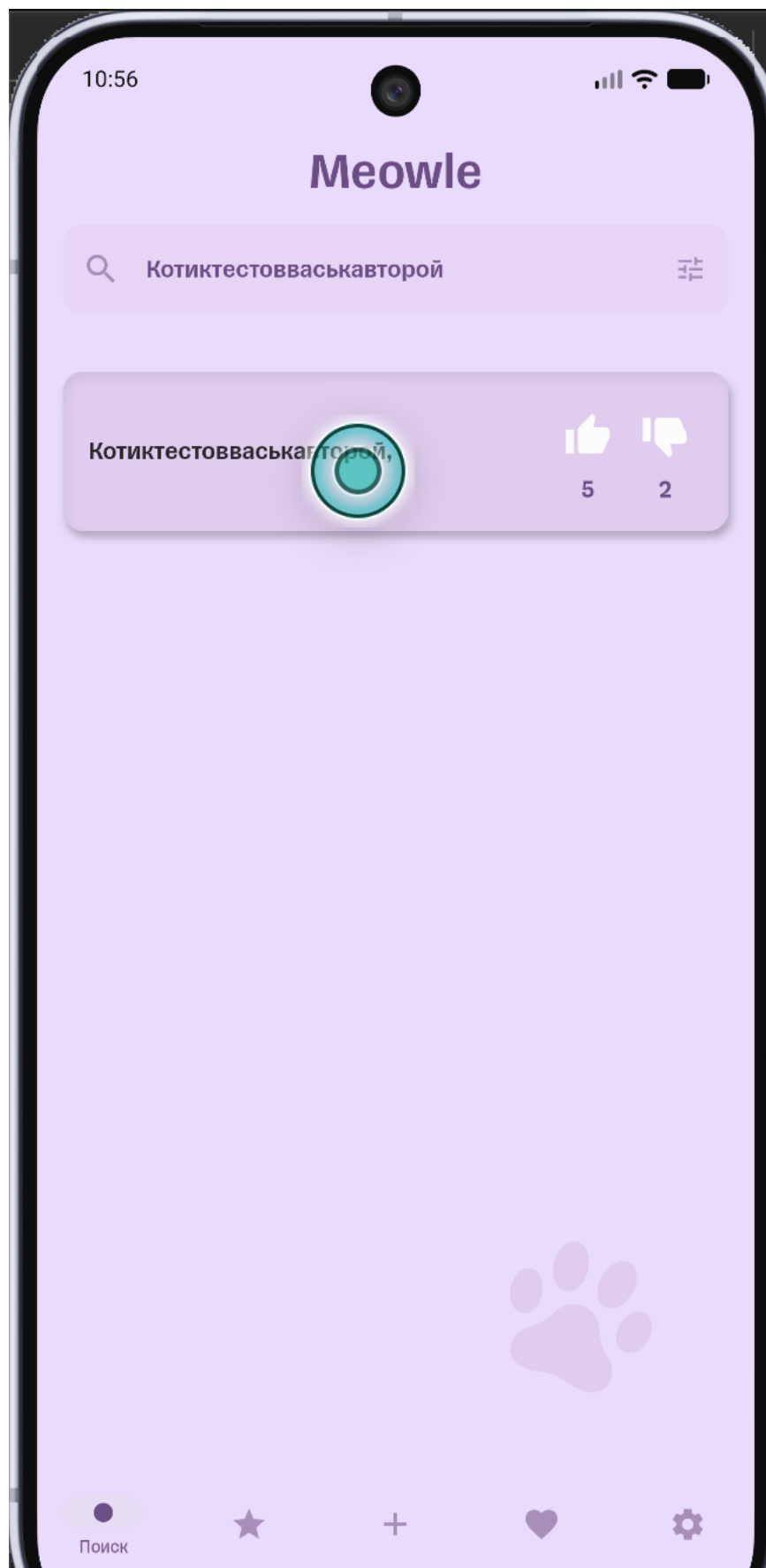


Рисунок 52 – Список результатов поиска до подмены

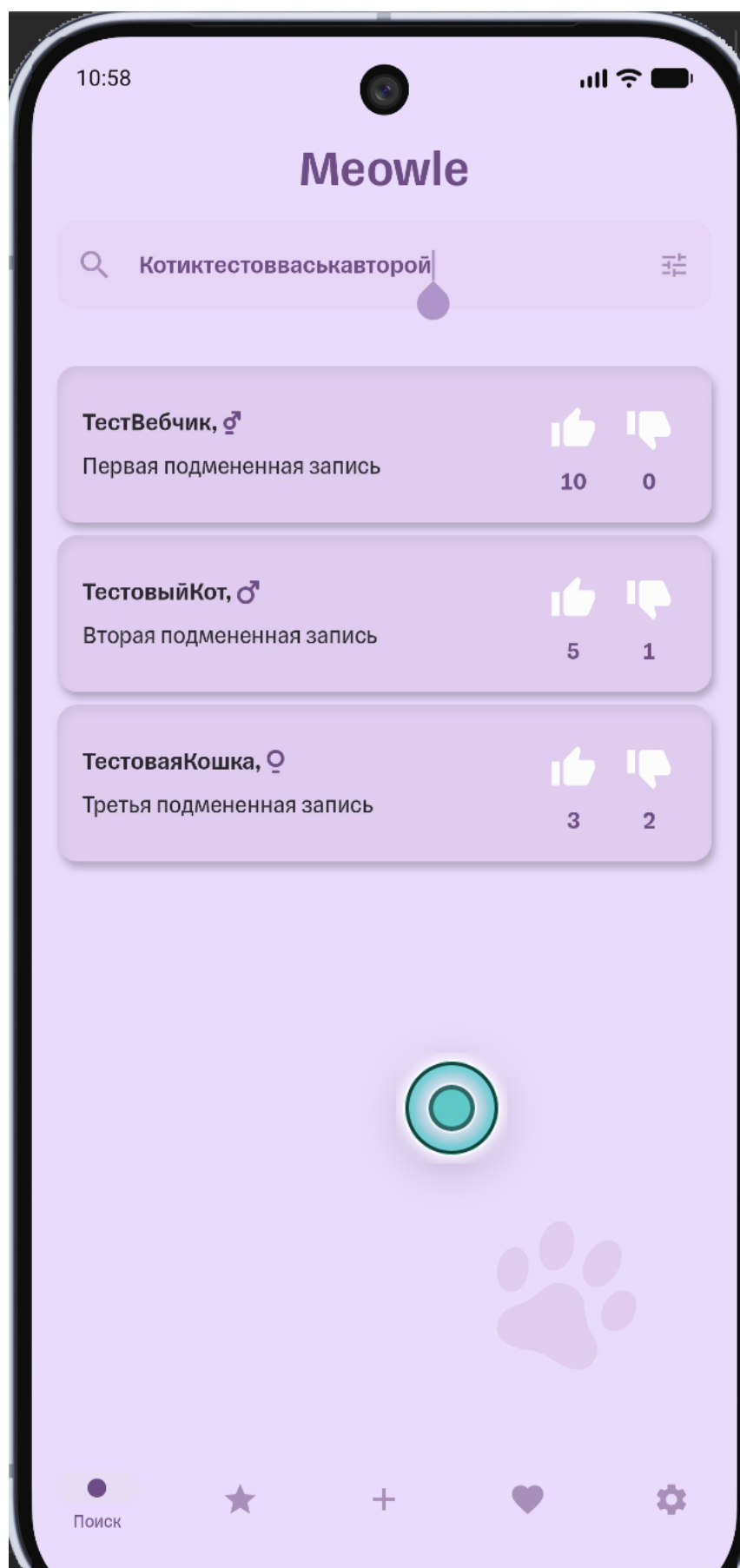


Рисунок 53 – Список результатов поиска после подмены

5.6 Эксперимент 4. Подмена списка фото или URL изображения

Исходное действие

В приложении открывается карточка котика с фотографиями. Приложение отправляет запрос: GET /api/photos/cats/68/photos

Исходный ответ

```
{
  "images": [
    "/photos/image-1780634233797.jpg"
  ]
}
```

Если у выбранного котика нет фото, исходный ответ может быть таким:

```
{
  "images": []
}
```

Подмененный ответ

Через Proxymap можно заменить список изображений:

```
{
  "images": [
    "/photos/image-1780634233797.jpg",
    "/photos/fake-client-image.jpg"
  ]
}
```

Также можно заменить существующий URL на несуществующий:

```
{
  "images": [
    "/photos/fake-client-image.jpg"
  ]
}
```

Таблица 23 – Результат

Пункт	Значение
Что изменилось в приложении	Приложение пытается загрузить изображения из подмененного списка
Изменилось ли состояние сервера	Нет
Как проверено	После отключения подмены список фото снова соответствует реальному ответу /api/photos/cats/68/photos
Вывод	Подмена URL влияет только на отображение в приложении. Новые фото на сервер не загружаются

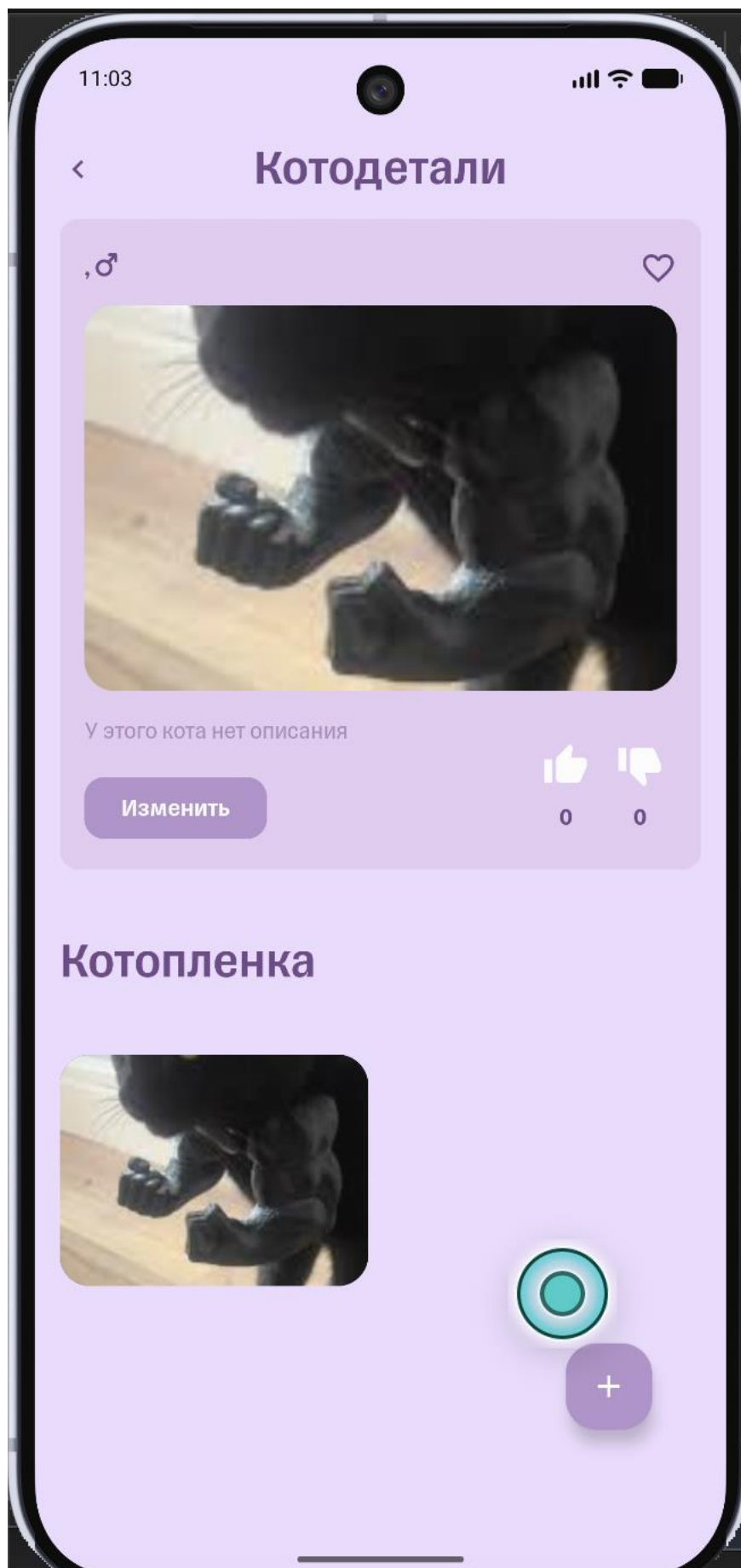


Рисунок 54 – Список фото до подмены

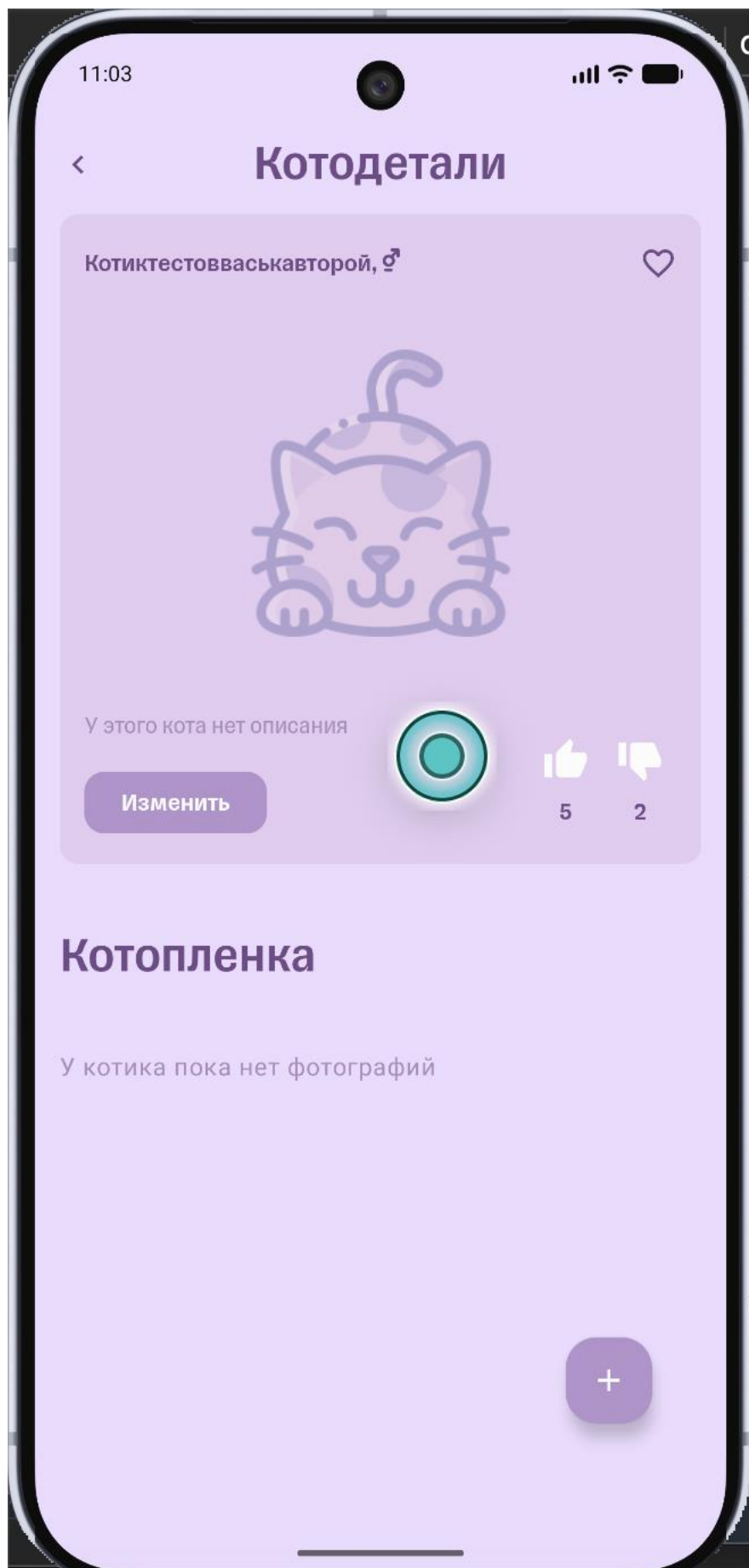


Рисунок 55 – Список фото или URL изображения после подмены

5.7 Общий вывод по изменению ответов

Во всех экспериментах изменения остались только на стороне клиента. Прокси изменял HTTP-ответ после того, как сервер уже сформировал данные, поэтому серверное состояние не менялось.

После отключения правил Proxypup и повторного выполнения тех же действий приложение снова получало реальные данные backend API. Это показывает, что мобильный клиент не должен считаться надежным источником данных: отображаемые значения можно изменить в сетевом ответе, а настоящим источником состояния должен быть сервер.

С точки зрения безопасности это не всегда означает уязвимость backend API, но показывает важный принцип: любые проверки, влияющие на права доступа, рейтинг, возможность редактирования или загрузки файлов, должны выполняться на сервере, а не только в мобильном приложении.

6. SECURITY-АНАЛИЗ

6.1 Цель этапа

Цель security-анализа - определить, какие проверки выполняются на сервере, а какие ограничения реализованы только на стороне мобильного приложения. Для проверки использовались Swagger UI, Proxymap Replay / Compose и ручные HTTP-запросы.

Таблица 24 – Тестовый котик

Поле	Значение
catId	68
name	Котиктестоввасьякавторой
description	Описание изменено через Proxymap
gender	unisex

Таблица 25 – Публичная запись

Поле	Значение
catId	59
name	Тестик
Исходное описание	Публичный тестик

6.2 Сводка найденных проблем

Таблица 26 – Сводка

#	Title	Severity	Направление
1	API-запросы выполняются без обязательной авторизации	High	Авторизация
2	Прямой доступ к объектам по id	Medium	IDOR / Object access
3	Возможность изменить описание чужой записи	High	Broken access control
4	Повторный лайк напрямую через API увеличивает рейтинг	Medium	Business logic
5	Невалидный gender принимается сервером	Medium	Валидация параметров
6	Ошибки валидации возвращаются как 200 OK	Low	HTTP-статусы
7	Некорректная обработка ошибок при загрузке файлов	Medium	Upload / Error handling
8	Нет заметного ограничения частоты запросов	Medium	Rate limit

6.3 Finding 1. API-запросы выполняются без обязательной авторизации

Title

API-запросы выполняются без обязательной авторизации.

Severity

High.

Preconditions

Атакующий знает адрес backend API:

http://144.31.255.0:3001

Дополнительный токен, cookie сессии или заголовок Authorization не требуются.

Открыть Proxyman и перехватить запросы мобильного приложения.

Проверить headers у запросов:

POST /api/core/cats/search

GET /api/core/cats/get-by-id?id=68

POST /api/core/cats/save-description

POST /api/likes/cats/68/likes

GET /api/likes/cats/rating

В запросах нет обязательного заголовка:

Authorization: Bearer ...

Запрос вручную через Proxyman Compose или curl.

curl -i 'http://144.31.255.0:3001/api/likes/cats/rating'

Actual result

Сервер возвращает данные без проверки авторизации:

HTTP/1.1 200 OK

Пример ответа:

```
{  
  "likes": [],  
  "dislikes": []  
}
```

Фактические списки likes и dislikes зависят от текущего состояния стенда.

Expected result

Сервер должен требовать авторизацию для действий, которые читают или изменяют пользовательские данные. Для неавторизованного запроса ожидаемый ответ:

HTTP/1.1 401 Unauthorized

или:

HTTP/1.1 403 Forbidden

В Proxyman видно, что запросы выполняются без Authorization, но сервер отвечает 200 OK.

```

Last login: Fri Jun  5 09:15:28 on ttys000
vasiliy@MacBook-Pro-Vasiliy ~ % curl -i 'http://144.31.255.0:3001/api/likes/cats/rati
ng'
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 2592
ETag: W/"a20-YdmLvGwVtBYTuXfN4VPP80bUCg"
Date: Sat, 06 Jun 2026 14:11:18 GMT
Connection: keep-alive

{"likes":[{"id":54,"name":"Тивирп","description":"Описание изменено через прямой API-запрос","tags":"","gender":"unisex","likes":230,"dislikes":1},{id":21,"name":"Турбопетров","description":"Новый текст","tags":"","gender":"unisex","likes":42,"dislikes":40},{id":30,"name":"Леопольд","description":"Леопольд вернулся. Удаляйтеааааа","tags":"","gender":"male","likes":30,"dislikes":1},{id":37,"name":"Бодя","description":"у уаа","tags":"","gender":"unisex","likes":21,"dislikes":19},{id":18,"name":"Обормотик","description":"Изменил","tags":"","gender":"unisex","likes":20,"dislikes":15},{id":47,"name":"Нии","description":"в","tags":"","gender":"unisex","likes":14,"dislikes":13},{id":42,"name":"Велик","description":"великий кот новгородский6","tags":"","gender":"male","likes":12,"dislikes":12},{id":20,"name":"Вадосик","description":"67","tags":"","gender":"male","likes":11,"dislikes":11},{id":29,"name":"Новыйбогдан","description":"yy","tags":"","gender":"unisex","likes":8,"dislikes":6},{id":68,"name":"Котиктестовваськавторой","description":"","tags":"","gender":"unisex","likes":5,"dislikes":2}],dislikes":[{"id":21,"name":"Турбопетров","description":"Новый текст","tags":"","gender":"unisex","likes":42,"dislikes":40},{id":37,"name":"Бодя","description":"у уаа","tags":"","gender":"unisex","likes":21,"dislikes":19},{id":18,"name":"Обормотик","description":"Изменил","tags":"","gender":"unisex","likes":20,"dislikes":15},{id":47,"name":"Нии","description":"в","tags":"","gender":"unisex","likes":14,"dislikes":13},{id":42,"name":"Велик","description":"великий кот новгородский6","tags":"","gender":"male","likes":12,"dislikes":12},{id":20,"name":"Вадосик","description":"67","tags":"","gender":"male","likes":11,"dislikes":11},{id":29,"name":"Новыйбогдан","description":"yy","tags":"","gender":"unisex","likes":8,"dislikes":6},{id":5,"name":"Буся","description":"Запись для ручного редактирования\\\"\\\"","tags":"","gender":"female","likes":4,"dislikes":4},{id":32,"name":"Неверный Генден","description":"Тестовое описание","tags":"","gender":"unisex","likes":4,"dislikes":4},{id":68,"name":"Котиктестовваськавторой","description":"","tags":"","gender":"unisex","likes":5,"dislikes":2}]}

```

Рисунок 56 – Запрос к API без заголовка Authorization

Recommendation

Добавить обязательную серверную авторизацию:

- выдавать пользователю токен после входа;
- проверять токен на каждом API-запросе;
- привязывать действия к конкретному пользователю;
- возвращать 401 Unauthorized, если пользователь не авторизован;
- возвращать 403 Forbidden, если пользователь авторизован, но не имеет прав на объект.

6.4. Finding 2. Прямой доступ к объектам по id

Title

Карточку котика можно получить напрямую по id.

Severity

Medium.

Preconditions

Атакующий знает или подбирает catId.

Steps to reproduce

Взять запрос открытия карточки из приложения:

GET /api/core/cats/get-by-id?id=68

Изменить параметр id на другой:

GET /api/core/cats/get-by-id?id=59

Отправить запрос через Proxymon Compose или curl.

Пример:

curl -i 'http://144.31.255.0:3001/api/core/cats/get-by-id?id=59'

Actual result

Сервер возвращает карточку объекта по указанному id:

```
{
  "cat": {
    "id": 59,
    "name": "Тестик",
    "description": "Публичный тестик",
    "tags": "",
    "gender": "unisex",
    "likes": 0,
    "dislikes": 0
  }
}
```

Expected result

Если карточки должны быть публичными, сервер может разрешать чтение. Но если объект связан с конкретным пользователем или содержит непубличные данные, сервер должен проверять права доступа.

Для закрытого объекта ожидаемый ответ:

HTTP/1.1 403 Forbidden

Evidence



```
[vasiliy@MacBook-Pro-Vasiliy ~ % curl -i 'http://144.31.255.0:3001/api/core/cats/get-by-id?id=59'
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 136
ETag: W/"88-XPzURSM6Y8gJxBE8U6V1H8/xwec"
Date: Sat, 06 Jun 2026 14:14:12 GMT
Connection: keep-alive

{"cat":{"id":59,"name":"Тестик","description":"Публичный тестик","tags":"","gender":"male","likes":0,"dislikes":0}}
```

Рисунок 57 – Получение карточки другого котика по прямому id

Recommendation

Не полагаться на скрытие id в интерфейсе. На сервере нужно проверять, какие объекты пользователь может просматривать. Если карточки являются публичными по бизнес-логике, это нужно явно описать в модели доступа.

6.5 Finding 3. Возможность изменить описание чужой записи

Title

Пользователь может изменить описание записи, которую он не создавал.

Severity

High.

Preconditions

Атакующий знает catId чужой записи и может отправить запрос к API.

Steps to reproduce

Найти чужую запись через поиск или прямой запрос карточки:

GET /api/core/cats/get-by-id?id=59

Сохранить исходное описание, чтобы восстановить его после проверки.

Отправить запрос изменения описания:

POST /api/core/cats/save-description

Content-Type: application/json

```
{
  "catId": 59,
  "catDescription": "Временная проверка доступа"
}
```

curl:

```
curl -i -X POST 'http://144.31.255.0:3001/api/core/cats/save-description' \
  -H 'Content-Type: application/json' \
  -d '{"catId":59,"catDescription":"Временная проверка доступа"}
```

Повторно открыть карточку:

GET /api/core/cats/get-by-id?id=59

После фиксации результата восстановить исходное описание:

```
{
  "catId": 59,
  "catDescription": "Публичный тестик"
}
```

Actual result

Сервер изменяет описание чужой записи и возвращает обновленный объект:

```
{
  "id": 59,
  "name": "Тестик",
  "description": "Временная проверка доступа",
  "tags": "",
  "gender": "unisex",
  "likes": 0,
  "dislikes": 0
}
```

Expected result

Сервер должен проверять владельца записи. Если пользователь не является владельцем объекта, ожидаемый ответ:

HTTP/1.1 403 Forbidden

Описание не должно изменяться.

Evidence

```
vasiliy@MacBook-Pro-Vasiliy ~ % curl -i -X POST 'http://144.31.255.0:3001/api/core/cats/save-description' \
  -H 'Content-Type: application/json' \
  -d '{"catId":59,"catDescription":"Временная проверка доступа"}
```



```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 147
ETag: W/"93-qQmWfmF+fhfdqQoTgmPMW0rE3XA"
Date: Sat, 06 Jun 2026 14:15:37 GMT
Connection: keep-alive

{"id":59,"name":"Тестик","description":"Временная проверка доступа","tags":"","gender":"male","likes":0,"dislikes":0}
```

Рисунок 58 – Изменение описания чужой записи через /api/core/cats/save-description

Recommendation

Добавить server-side access control:

- хранить владельца записи;
- проверять владельца перед изменением описания;
- запрещать изменение чужих записей;
- логировать попытки изменения чужих объектов;
- возвращать 403 Forbidden при отсутствии прав.

6.6 Finding 4. Повторный лайк напрямую через API увеличивает рейтинг Title

Один и тот же запрос лайка можно повторять, увеличивая рейтинг.

Severity

Medium.

Preconditions

Атакующий знает catId и endpoint лайка.

Steps to reproduce

Отправить запрос лайка:

POST /api/likes/cats/68/likes

Content-Type: application/json

```
{
  "dislike": false,
  "like": true
}
```

Повторить этот же запрос несколько раз.

```
curl -i -X POST 'http://144.31.255.0:3001/api/likes/cats/68/likes' \
```

```
-H 'Content-Type: application/json' \
```

```
-d '{"dislike":false,"like":true}'
```

Проверить ответ после каждого повтора.

Actual result

Счетчик likes увеличивается при повторе одного и того же действия:

Фактическое значение likes зависит от количества повторов.

Expected result

Лайк должен быть идемпотентным. Один пользователь должен иметь возможность поставить только один лайк одной записи. Повторный запрос должен:

- не увеличивать счетчик повторно;
- возвращать текущее состояние голоса;
- либо возвращать ошибку, если повтор запрещен бизнес-логикой.

Evidence

```

vasiliy@MacBook-Pro-Vasiliy ~ % curl -i -X POST 'http://144.31.255.0:3001/api/likes/cats/68/likes' \
-H 'Content-Type: application/json' \
-d '{"dislike":false,"like":true}'

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 133
ETag: W/"85-RddRTY1HxJPoImNkmBFYSk+tDXM"
Date: Sat, 06 Jun 2026 14:17:11 GMT
Connection: keep-alive

{"id":68,"name":"Котиктестовваськавторой","description":"","tags":"","gender":"unisex",
,"likes":6,"dislikes":2}%
vasiliy@MacBook-Pro-Vasiliy ~ % curl -i -X POST 'http://144.31.255.0:3001/api/likes/cats/68/likes' \
-H 'Content-Type: application/json' \
-d '{"dislike":false,"like":true}'

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 133
ETag: W/"85-GZIrJOWXRtJL77Atp4SAHWUUhC"
Date: Sat, 06 Jun 2026 14:17:14 GMT
Connection: keep-alive

{"id":68,"name":"Котиктестовваськавторой","description":"","tags":"","gender":"unisex",
,"likes":7,"dislikes":2}%
vasiliy@MacBook-Pro-Vasiliy ~ % █

```

Рисунок 59 – Повторный replay лайка с увеличением счетчика

Recommendation

Перед изменением счетчика сервер должен проверять голос пользователя:

- хранить связь `userId + catId + vote`;
- делать операцию идемпотентной;
- не увеличивать счетчик при повторном таком же голосе;
- разрешать переключение с лайка на дизлайк только как изменение существующего

голоса.

6.7. Finding 5. Невалидный gender принимается сервером

Title

Сервер принимает невалидное значение `gender` и создает запись.

Severity

Medium.

Preconditions

Атакующий может отправить запрос добавления котика.

Steps to reproduce

Отправить запрос:

POST `/api/core/cats/add`

Content-Type: `application/json`

```

{
  "cats": [
    {
      "name": "Проверка ПолДва ",
      "gender": "dragon",

```



```

        "description": "Невалидный gender"
    }
  ]
}
curl -i -X POST 'http://144.31.255.0:3001/api/core/cats/add' \
-H 'Content-Type: application/json' \
-d '{"cats":[{"name":"Проверка ПолДва","gender":"dragon","description":"Невалидный
gender"}]}'

```

Actual result

Сервер создает запись и меняет невалидный gender на unisex:

```

{
  "cats": [
    {
      "id": 70,
      "name": "Проверка ПолДва",
      "description": "Невалидный gender",
      "tags": "",
      "gender": "unisex",
      "likes": 0,
      "dislikes": 0
    }
  ]
}

```

Expected result

Сервер должен отклонять неизвестные значения gender.

Ожидаемый ответ:

HTTP/1.1 400 Bad Request

Пример корректного сообщения:

```

{
  "error": "Invalid gender",
  "allowedValues": ["male", "female", "unisex"]
}

```

Evidence

```

vasiliy@MacBook-Pro-Vasiliy ~ % curl -i -X POST 'http://144.31.255.0:3001/api/core/ca
ts/add' \
-H 'Content-Type: application/json' \
[ -d '{"cats":[{"name":"Проверка Полдва","gender":"dragon","description":"Невалидный
gender"}]}'

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 154
ETag: W/"9a-RPLwcuHppPR+6DPkprX1e3WUNgs"
Date: Sat, 06 Jun 2026 14:19:44 GMT
Connection: keep-alive

{"cats":[{"id":72,"name":"Проверка Полдва","description":"Невалидный gender","tags":"
","gender":"unisex","likes":0,"dislikes":0}]}%

```

Рисунок 60 – Создание записи с gender: "dragon"

Recommendation

Добавить строгую server-side валидацию enum-полей:

- разрешить только male, female, unisex;
- не исправлять невалидное значение молча;

- возвращать 400 Bad Request;
- описать допустимые значения в Swagger.

6.8 Finding 6. Ошибки валидации возвращаются как 200 OK

Title

При ошибке валидации сервер возвращает 200 OK и кладет ошибку внутрь JSON.

Severity

Low.

Preconditions

Атакующий может отправить некорректный запрос добавления котика.

Steps to reproduce

Отправить запрос с пустым списком имен или некорректной структурой:

POST /api/core/cats/add

Content-Type: application/json

```
{
  "cats": []
}
```

Также можно повторить проверку с дубликатом имени:

```
{
  "cats": [
    {
      "name": "КотикТестовВаськаВторой",
      "gender": "unisex",
      "description": "Дубликат имени"
    }
  ]
}
```

Actual result

Сервер возвращает ответ с ошибкой внутри тела:

```
{
  "cats": [
    {
      "errorDescription": "Передан пустой список имён"
    }
  ]
}
```

При этом HTTP-статус может оставаться:

HTTP/1.1 200 OK

Expected result

Для ошибок валидации должен использоваться статус:

HTTP/1.1 400 Bad Request

Тело ответа должно быть единообразным:

```
{
  "error": "Validation error",
  "details": [
    {
      "field": "cats",
      "message": "Передан пустой список имен"
    }
  ]
}
```

Evidence

```

vasiliy@MacBook-Pro-Vasiliy ~ % curl -i -X POST 'http://144.31.255.0:3001/api/core/cats/add' \
  -H 'Content-Type: application/json' \
  -d '{"cats":[]}'
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 83
ETag: W/"53-bpF0d+0ZqYKwMCdy2Hmiuv5JQ90"
Date: Sat, 06 Jun 2026 14:25:59 GMT
Connection: keep-alive

{"cats":[{"errorDescription":"Передан пустой список имён"}]}%
vasiliy@MacBook-Pro-Vasiliy ~ %

```

Рисунок 61 – Ошибка валидации в body при статусе 200 OK

Recommendation

Исправить HTTP-статусы:

- 400 Bad Request для ошибок валидации;
- 404 Not Found для несуществующего объекта;
- 409 Conflict для дубликата имени;
- единый JSON-формат ошибок для всех endpoint.

6.9 Finding 7. Некорректная обработка ошибок при загрузке файлов

Title

Endpoint загрузки файла возвращает HTML-ошибки и 500 Internal Server Error при некорректном multipart-запросе.

Severity

Medium.

Preconditions

Атакующий может отправить запрос на загрузку файла:

POST /api/photos/cats/{catId}/upload

Steps to reproduce

Отправить корректный запрос загрузки изображения:

POST /api/photos/cats/68/upload

Content-Type: multipart/form-data

Multipart part:

Part: file

Content Type: image/jpeg

File Name: file.jpg

Value: Binary File

Затем изменить multipart:

указать неправильное имя part, например name вместо file;

указать некорректный Content-Type;

отправить файл с неправильным расширением;

отправить поврежденное тело multipart.

Actual result

Вместо стабильного JSON-ответа сервер возвращает HTML-страницу ошибки:

```
<pre>Bad Request</pre>
```

или:

```
<pre>Internal Server Error</pre>
```

Также возможен HTTP-статус:

HTTP/1.1 500 Internal Server Error

Expected result

Сервер должен возвращать контролируемую ошибку в JSON-формате:
 HTTP/1.1 400 Bad Request
 Content-Type: application/json
 {
 "error": "Invalid upload",
 "message": "Expected multipart field 'file' with image/jpeg or image/png"
 }
 500 Internal Server Error не должен возникать из-за пользовательского ввода.
 Evidence

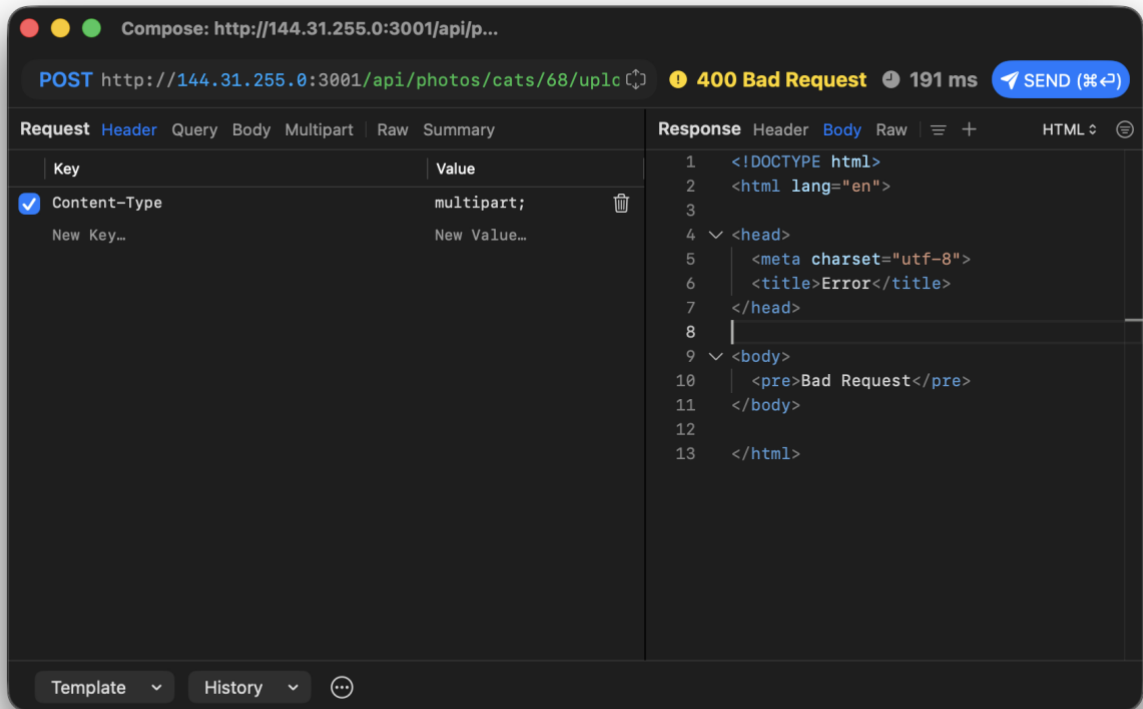


Рисунок 62 – 400 Bad Request при некорректной загрузке файла

Recommendation

Улучшить обработку upload:

строго проверять имя multipart-поля file;

проверять MIME type и расширение;

проверять фактическое содержимое файла, а не только расширение;

ограничить размер файла;

возвращать 400 Bad Request с JSON-ошибкой;

не отдавать HTML-страницы ошибок API-клиенту.

6.10 Finding 8. Нет заметного ограничения частоты запросов

Title

API не ограничивает частоту повторных запросов.

Severity

Medium.

Preconditions

Атакующий может отправлять повторные запросы к backend API.

Steps to reproduce

В Proxymap выбрать запрос, который меняет состояние, например:

POST /api/likes/cats/68/likes

Несколько раз нажать Replay.
 Повторить проверку для запросов:
 POST /api/core/cats/search
 POST /api/core/cats/add
 POST /api/core/cats/save-description
 Actual result

Сервер продолжает обрабатывать повторные запросы. При повторе лайка состояние рейтинга может изменяться несколько раз.

Expected result

Сервер должен ограничивать частоту запросов:

по пользователю;

по IP-адресу;

по endpoint;

по типу операции.

При превышении лимита ожидаемый ответ:

HTTP/1.1 429 Too Many Requests

Evidence

#	URL	Client	Method	Status	Time	Duration	Re
247	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	22:38:58.895	189 ms	
306	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:35:35.678	209 ms	
314	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:47.527	189 ms	
322	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:49.476	186 ms	
330	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:51.130	249 ms	
338	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:52.942	185 ms	
346	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:55.543	480 ms	
354	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:37:00.328	196 ms	
356	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:37:00.961	192 ms	
352	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:58.468	285 ms	
350	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:57.786	186 ms	
348	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:56.999	186 ms	
344	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:55.102	249 ms	
342	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:54.486	238 ms	
340	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:53.435	228 ms	
336	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:52.493	196 ms	
334	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:52.089	193 ms	
332	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:51.656	199 ms	
328	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:50.725	194 ms	
326	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:50.343	182 ms	
324	http://144.31.255.0:3001/api/likes/cats/rating	netsimd	GET	200	23:36:49.826	202 ms	

Рисунок 62 – несколько повторных replay-запросов без 429 Too Many Requests

Recommendation

Добавить rate limiting:

- ограничить частоту изменения рейтинга;
- ограничить частоту создания котиков;
- ограничить частоту загрузки файлов;
- добавить защиту от автоматического перебора catId;
- возвращать 429 Too Many Requests при превышении лимитов.

6.11 Общий вывод по security-анализу

Основная проблема стенда заключается в том, что backend API доверяет клиенту больше, чем должен. Мобильное приложение может скрывать часть действий в интерфейсе, но запросы можно повторить или изменить через Proxymun, Postman, Insomnia или curl.

Критичные проверки должны выполняться на сервере:

- авторизация пользователя;
- проверка владельца объекта;
- проверка допустимых значений параметров;
- защита от повторных действий;
- корректная обработка ошибок;
- ограничения на частоту запросов;
- проверка загружаемых файлов.

Мобильный клиент нельзя считать надежной границей безопасности, потому что пользователь может перехватить, повторить и изменить любой HTTP-запрос.